

Data Structures & Algorithms in Python

A Complete Reference for C++ Programmers

Arrays · Linked Lists · Stacks · Queues · Trees · Heaps · Hash Tables · Tries
Segment Trees · Fenwick Trees · Sparse Tables · DSU
Graphs · Sorting · Searching · Backtracking · Dynamic Programming
Greedy · Divide & Conquer · String Algorithms · Mathematical Algorithms · Bit Manipulation

Every Algorithm · Every Data Structure · Every Complexity

Contents

Part I

Foundations

Chapter 1

Python for C++ Programmers

This guide assumes full fluency in C++ DSA. Python is dynamically typed, garbage-collected, and has arbitrary-precision integers — no overflow, no manual memory. The trade-off is speed: expect Python to be 10–100× slower than C++.

1.1 C++ ↔ Python Quick-Reference

	Concept	C++	Python 3
Dynamic array	<code>vector<int></code>		<code>list</code>
Stack	<code>stack<T></code>		<code>list</code> or <code>collections.deque</code>
Queue	<code>queue<T></code>		<code>collections.deque</code>
Deque	<code>deque<T></code>		<code>collections.deque</code>
Priority queue (max)	<code>priority_queue<T></code>		<code>heapq</code> (negate for max-heap)
Ordered map	<code>map<K,V></code>		<code>dict</code> (insertion-ordered 3.7+)
Unordered map	<code>unordered_map<K,V></code>		<code>dict</code>
Ordered set	<code>set<T></code>		<code>sortedcontainers.SortedList</code>
Unordered set	<code>unordered_set<T></code>		<code>set</code>
Pair	<code>pair<A,B></code>		<code>tuple</code>
Tuple	<code>tuple<...></code>		<code>tuple</code>
String	<code>std::string</code> (mutable)		<code>str</code> (immutable)
Null pointer	<code>nullptr</code>		<code>None</code>
Boolean literals	<code>true</code> / <code>false</code>		<code>True</code> / <code>False</code>
Integer overflow	Yes — use <code>long long</code>		Never (arbitrary precision)
Infinity	<code>INT_MAX</code> , <code>LLONG_MAX</code>		<code>float('inf')</code>
Auto type	<code>auto x = ...</code>		No declaration needed
Lambda	<code>[](int x){return x*2;}</code>		<code>lambda x: x*2</code>
Range-based for	<code>for(auto x : vec)</code>		<code>for x in lst</code>
Ternary	<code>cond ? a : b</code>		<code>a if cond else b</code>
Sort (custom cmp)	<code>sort(a,b,cmp)</code>		<code>sorted(lst, key=func)</code>
Modular inverse	Manual (ext. Euclidean)		<code>pow(a, -1, mod)</code>
GCD	<code>__gcd(a,b)</code>		<code>math.gcd(a,b)</code>
Swap	<code>swap(a,b)</code>		<code>a, b = b, a</code>
Bit operations	<code>&</code> , <code> </code> , <code>^</code> , <code>~</code> , <code>«</code> , <code>»</code>		same operators
Input	<code>cin >> x</code>		<code>x = int(input())</code>

1.2 Essential Python Idioms

```
1  #           Swap (no temp variable)
2  a, b = b, a
```

```

3
4 #           Multiple assignment

5 x, y, z = 1, 2, 3
6 first, *rest = [1, 2, 3, 4]      # first=1, rest=[2,3,4]
7 *init, last = [1, 2, 3, 4]      # init=[1,2,3], last=4
8
9 #           List comprehension (replaces C++ for-loop building vector)
10 squares = [x**2 for x in range(10)]
11 evens    = [x for x in range(20) if x % 2 == 0]
12 flat     = [item for sub in matrix for item in sub] # flatten 2D
13
14 #           Dict / Set comprehensions

15 freq     = {ch: s.count(ch) for ch in set(s)}
16 uniq     = {x for x in arr}
17
18 #           Chained comparisons

19 if 0 <= i < n:    pass          # C++: i >= 0 && i < n
20
21 #           Enumerate (index + value)

22 for i, val in enumerate(arr):
23     pass
24
25 #           Zip (iterate two sequences in parallel)

26 for a, b in zip(list1, list2):
27     pass
28 pairs = list(zip(keys, values))
29
30 #           Negative indexing

31 arr[-1]          # last element
32 arr[-2]          # second to last
33
34 #           Slicing arr[start:stop:step]

35 arr[1:4]          # elements at index 1, 2, 3
36 arr[::-1]         # reversed copy
37 arr[::2]          # every other element
38 arr[i:j] = []     # delete elements i to j-1 in-place
39
40 #           String building (use join, NOT +=)

41 # SLOW: result = ""; for ch in s: result += ch    # O(n^2)
42 # FAST:
43 parts = []
44 for ch in s:

```

```

45     parts.append(ch)
46 result = "".join(parts)      # O(n)
47
48 #           Ternary
49
49 result = x if condition else y
50
51 #           Truthiness
52
52 # Falsy: None, 0, 0.0, "", [], {}, set()
53 # All others: truthy
54
55 #           Unpacking in function args
56
56 def f(a, b, c): pass
57 args = [1, 2, 3]
58 f(*args)                # positional unpacking
59 kwargs = {'a':1, 'b':2, 'c':3}
60 f(**kwargs)             # keyword unpacking
61
62 #           walrus operator (:=) assigns and evaluates
63
63 if (n := len(arr)) > 10:
64     print(f"Array has {n} elements")

```

1.3 Standard Library for DSA

```

1  import sys, math, heapq, bisect
2  from collections import defaultdict, Counter, deque, OrderedDict
3  from functools import lru_cache, cache, reduce
4  from itertools import permutations, combinations,
5      combinations_with_replacement, \
6      product, chain, accumulate
7
8  #           sys
9
9  sys.setrecursionlimit(300000)      # essential for deep DFS
10 INF = float('inf')
11 NINF = float('-inf')
12 input = sys.stdin.readline          # fast input for competitive
13                                     programming
14
15 #           math
16
16 math.gcd(12, 8)                     # 4
17 math.lcm(4, 6)                      # 12 (Python 3.9+)
18 math.isqrt(17)                      # 4 (integer sqrt, no float rounding)
19 math.log2(1024)                     # 10.0
20 math.comb(10, 3)                    # 120 (nC_r, exact integer)
21 math.factorial(10)                  # 3628800

```

```

20 math.ceil(3.2)           # 4
21 math.floor(3.9)          # 3
22 pow(2, 10, 1000)         # modular exponentiation: 210 mod 1000 =
    24
23 pow(3, -1, 7)           # modular inverse: 3-1 mod 7 = 5
24
25 #             heapq (min-heap)

26 h = []
27 heapq.heappush(h, 5)
28 top = heapq.heappop(h)   # smallest element
29 heapq.heapify(lst)       # O(n) in-place
30 heapq.nlargest(k, lst)   # k largest in O(n log k)
31 heapq.nsmallest(k, lst)  # k smallest
32 # Max-heap trick: push negative values
33 heapq.heappush(h, -val)
34 max_val = -heapq.heappop(h)
35
36 #             bisect (sorted list binary search)

37 arr = [1, 3, 5, 7, 9]
38 bisect.bisect_left(arr, 5) # 2 (leftmost insert position)
39 bisect.bisect_right(arr, 5) # 3 (rightmost insert position)
40 bisect.insort(arr, 6)      # insert 6 maintaining sort: O(n)
41
42 #             defaultdict

43 graph = defaultdict(list) # adjacency list
44 freq = defaultdict(int)   # frequency map (default 0)
45 dp = defaultdict(lambda: INF) # custom default
46
47 #             Counter

48 c = Counter("banana")     # {'a':3, 'n':2, 'b':1}
49 c.most_common(2)          # [('a',3), ('n',2)]
50 c['z']                    # 0 (missing key safe)
51 c1 + c2                   # union of counters
52 c1 - c2                   # difference
53
54 #             deque

55 dq = deque([1, 2, 3])
56 dq.appendleft(0)          # O(1) -- front push
57 dq.popleft()              # O(1) -- front pop
58 dq.rotate(k)              # rotate right by k
59 dq.maxlen                 # fixed-size sliding window: deque(
    maxlen=k)
60
61 #             itertools

62 list(permutations([1,2,3])) # all 3! orderings

```

```
63 list(combinations([1,2,3], 2))    # [(1,2),(1,3),(2,3)]
64 list(product([0,1], repeat=3))    # all 2^3 binary strings
65 list(accumulate([1,2,3,4]))      # [1, 3, 6, 10] (prefix sums)
66 list(chain([1,2], [3,4]))         # [1, 2, 3, 4]
67
68 #          lru_cache for memoization

69 @lru_cache(maxsize=None)
70 def fib(n):
71     if n <= 1: return n
72     return fib(n-1) + fib(n-2)
73
74 # Python 3.9+ shorthand
75 @cache
76 def fib(n):
77     if n <= 1: return n
78     return fib(n-1) + fib(n-2)
```

Recursion Limit

Python's default recursion limit is 1000. For DFS on large graphs or deep DP recursion, always add `sys.setrecursionlimit(300000)` at the top of your file. Alternatively, convert recursive DFS to iterative with an explicit stack.

Chapter 2

Complexity Analysis

2.1 Big-O Hierarchy

$$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

Practical rule of thumb for competitive programming: at 10^8 operations per second, $n \leq 10^8$ tolerates $O(n)$; $n \leq 10^4$ tolerates $O(n^2)$; $n \leq 500$ tolerates $O(n^3)$; $n \leq 20$ tolerates $O(2^n)$; $n \leq 12$ tolerates $O(n!)$.

2.2 Python Built-in Complexity Reference

Operation	Container	Avg	Worst
<code>lst[i]</code>	list	$O(1)$	$O(1)$
<code>lst.append(x)</code>	list	$O(1)$ amort	$O(n)$
<code>lst.pop()</code>	list	$O(1)$	$O(1)$
<code>lst.pop(0)</code>	list	$O(n)$	$O(n)$
<code>lst.insert(i,x)</code>	list	$O(n)$	$O(n)$
<code>x in lst</code>	list	$O(n)$	$O(n)$
<code>lst.sort()</code>	list	$O(n \log n)$	$O(n \log n)$
<code>lst[a:b]</code>	list	$O(b-a)$	$O(n)$
<code>d[k]</code>	dict	$O(1)$	$O(n)$
<code>d[k] = v</code>	dict	$O(1)$	$O(n)$
<code>k in d</code>	dict	$O(1)$	$O(n)$
<code>del d[k]</code>	dict	$O(1)$	$O(n)$
<code>x in s</code>	set	$O(1)$	$O(n)$
<code>s.add(x)</code>	set	$O(1)$	$O(n)$
<code>s.remove(x)</code>	set	$O(1)$	$O(n)$
<code>s1 s2 (union)</code>	set	$O(m+n)$	—
<code>s1 & s2 (inter.)</code>	set	$O(\min(m, n))$	—
<code>dq.append()</code>	deque	$O(1)$	$O(1)$
<code>dq.appendleft()</code>	deque	$O(1)$	$O(1)$
<code>dq.pop()</code>	deque	$O(1)$	$O(1)$
<code>dq.popleft()</code>	deque	$O(1)$	$O(1)$
<code>heapq.heappush()</code>	heap	$O(\log n)$	$O(\log n)$
<code>heapq.heappop()</code>	heap	$O(\log n)$	$O(\log n)$
<code>heapq.heapify()</code>	heap	$O(n)$	$O(n)$
<code>bisect.bisect()</code>	sorted list	$O(\log n)$	$O(\log n)$
<code>str + str</code>	str	$O(n)$	$O(n)$
<code>"".join(lst)</code>	str	$O(n)$	$O(n)$
<code>len(container)</code>	any	$O(1)$	$O(1)$

Part II

Linear Data Structures

Chapter 3

Arrays and Strings

3.1 Python Lists — Dynamic Arrays

```
1  # Declaration
2  arr = []                                # empty, like vector<int>
3  arr = [0] * n                          # n zeros
4  arr = list(range(n))                   # [0, 1, ..., n-1]
5  matrix = [[0]*cols for _ in range(rows)] # 2D --- NOT [[0]*cols]*
      rows !
6
7  # Core operations
8  arr.append(x)                          # push_back O(1) amortised
9  arr.pop()                              # pop_back O(1)
10 arr.pop(i)                             # erase(it) O(n)
11 arr.insert(i, x)                       # insert at i O(n)
12 arr.remove(x)                          # erase first x O(n)
13 arr.index(x)                           # first index of x O(n); raises ValueError if
      absent
14 x in arr                               # O(n) search; use set for O(1)
15 arr.sort()                             # in-place Timsort O(n log n)
16 arr.sort(reverse=True)
17 arr.sort(key=lambda x: x[1])           # sort by second element
18 sorted(arr)                            # returns NEW sorted list (original unchanged)
      )
19 arr.reverse()                          # in-place O(n)
20 arr.count(x)                           # count occurrences O(n)
21 arr.copy()                             # shallow copy O(n)
22 arr[:]                                 # also shallow copy
23 arr.clear()                            # O(n)
24 arr.extend(other)                      # append all elements of other O(k)
```

2D Array Pitfall

`[[0]*cols]*rows` creates *rows* references to the *same* inner list — modifying one row modifies all. Always use `[[0]*cols for _ in range(rows)]`.

3.2 Strings

```
1  s = "hello world"
2
3  # Strings are IMMUTABLE -- to mutate, convert to list first
4  lst = list(s)
5  lst[0] = 'H'
6  s = "".join(lst)
7
8  # Common methods
```

```

9 s.upper()           # "HELLO WORLD"
10 s.lower()          # "hello world"
11 s.strip()          # strip leading/trailing whitespace
12 s.lstrip(); s.rstrip()
13 s.split()           # split by whitespace -> list
14 s.split(',')        # split by delimiter
15 ", ".join(["a", "b"]) # "a, b"
16 s.replace('l', 'r') # replace ALL occurrences
17 s.find('lo')        # index of first match, -1 if missing
18 s.rfind('lo')       # last match
19 s.count('l')        # count non-overlapping occurrences
20 s.startswith('he')  # bool
21 s.endswith('ld')    # bool
22 s.isalpha()         # all letters?
23 s.isdigit()         # all digits?
24 s.isalnum()         # all alphanumeric?
25 s.isspace()         # all whitespace?
26 s.zfill(5)          # zero-pad: "007"
27 ord('a')            # 97 (like (int)'a' in C++)
28 chr(97)             # 'a' (like (char)97 in C++)
29 s[i:j]              # substring, O(j-i)
30 s[::-1]             # reverse string

```

3.3 Two Pointers

```

1 # Two Sum in Sorted Array TC: O(n) SC: O(1)

2 def two_sum_sorted(arr, target):
3     lo, hi = 0, len(arr) - 1
4     while lo < hi:
5         s = arr[lo] + arr[hi]
6         if s == target: return [lo, hi]
7         elif s < target: lo += 1
8         else: hi -= 1
9     return []

10
11 # Three Sum (unique triplets summing to 0) O(n^2)

12 def three_sum(nums):
13     nums.sort()
14     result = []
15     for i in range(len(nums) - 2):
16         if i > 0 and nums[i] == nums[i-1]:
17             continue
18         lo, hi = i + 1, len(nums) - 1
19         while lo < hi:
20             s = nums[i] + nums[lo] + nums[hi]
21             if s == 0:
22                 result.append([nums[i], nums[lo], nums[hi]])
23                 while lo < hi and nums[lo] == nums[lo+1]: lo += 1
24                 while lo < hi and nums[hi] == nums[hi-1]: hi -= 1
25                 lo += 1; hi -= 1
26             elif s < 0: lo += 1
27             else: hi -= 1
28     return result

```

```

29
30 #           Container with Most Water           O(n)

31 def max_water(height):
32     lo, hi = 0, len(height) - 1
33     res = 0
34     while lo < hi:
35         res = max(res, min(height[lo], height[hi]) * (hi - lo))
36         if height[lo] < height[hi]: lo += 1
37         else: hi -= 1
38     return res
39
40 #           Trapping Rain Water           O(n)

41 def trap(height):
42     n = len(height)
43     if n == 0: return 0
44     left_max = [0] * n
45     right_max = [0] * n
46     left_max[0] = height[0]
47     for i in range(1, n):
48         left_max[i] = max(left_max[i-1], height[i])
49     right_max[n-1] = height[n-1]
50     for i in range(n-2, -1, -1):
51         right_max[i] = max(right_max[i+1], height[i])
52     water = 0
53     for i in range(n):
54         water += min(left_max[i], right_max[i]) - height[i]
55     return water
56
57 # O(1) space two-pointer version
58 def trap_two_ptr(height):
59     lo, hi = 0, len(height) - 1
60     left_max = right_max = 0
61     water = 0
62     while lo <= hi:
63         if height[lo] <= height[hi]:
64             if height[lo] >= left_max: left_max = height[lo]
65             else: water += left_max - height[
66 lo]
67             lo += 1
68         else:
69             if height[hi] >= right_max: right_max = height[hi]
70             else: water += right_max - height[
71 hi]
72             hi -= 1
73     return water

```

3.4 Sliding Window

```

1 #           Fixed Window: Max Sum of Subarray Size k           O(n)
2
3 def max_sum_k(arr, k):

```

```

3     window = sum(arr[:k])
4     best = window
5     for i in range(k, len(arr)):
6         window += arr[i] - arr[i-k]
7         best = max(best, window)
8     return best
9
10    #           Variable Window: Longest Substring Without Repeating
11
12    def longest_unique_substr(s):           # TC: O(n)   SC: O(1)
13        seen = {}
14        lo = best = 0
15        for hi, ch in enumerate(s):
16            if ch in seen and seen[ch] >= lo:
17                lo = seen[ch] + 1
18            seen[ch] = hi
19            best = max(best, hi - lo + 1)
20        return best
21
22    #           Minimum Window Substring           O(|s|+|t|)
23
24    from collections import Counter
25    def min_window(s, t):
26        need = Counter(t)
27        have = total = 0
28        total = len(need)
29        window = {}
30        lo = 0
31        res_len = float('inf')
32        res = ""
33        for hi, ch in enumerate(s):
34            window[ch] = window.get(ch, 0) + 1
35            if ch in need and window[ch] == need[ch]:
36                have += 1
37            while have == total:
38                if hi - lo + 1 < res_len:
39                    res_len = hi - lo + 1
40                    res = s[lo:hi+1]
41                window[s[lo]] -= 1
42                if s[lo] in need and window[s[lo]] < need[s[lo]]:
43                    have -= 1
44                lo += 1
45            return res
46
47    #           Sliding Window Maximum (deque)           O(n)
48
49    from collections import deque
50    def max_sliding_window(nums, k):
51        dq = deque()           # stores indices, decreasing values
52        result = []
53        for i, x in enumerate(nums):
54            # remove indices out of window
55            while dq and dq[0] < i - k + 1:
56                dq.popleft()
57            # remove smaller elements from back
58            while dq and nums[dq[-1]] < x:
59                dq.pop()

```

```

57         dq.append(i)
58         if i >= k - 1:
59             result.append(nums[dq[0]])
60     return result

```

3.5 Prefix Sums

```

1  #           1D Prefix Sum

2  def build_prefix(arr):                # TC: O(n)   SC: O(n)
3      n = len(arr)
4      prefix = [0] * (n + 1)
5      for i in range(n):
6          prefix[i+1] = prefix[i] + arr[i]
7      return prefix
8  # Range sum [l, r] in O(1)
9  def range_sum(prefix, l, r): return prefix[r+1] - prefix[l]
10
11 # Using itertools.accumulate (Pythonic)
12 from itertools import accumulate
13 prefix = list(accumulate(arr, initial=0)) # [0, a0, a0+a1, ...]
14
15 #           Subarray Sum Equals k           O(n)

16 def subarray_sum_k(nums, k):
17     count = curr = 0
18     freq = {0: 1}
19     for x in nums:
20         curr += x
21         count += freq.get(curr - k, 0)
22         freq[curr] = freq.get(curr, 0) + 1
23     return count
24
25 #           Product of Array Except Self           O(n)   SC: O(1) output
26 #           excl
27 def product_except_self(nums):
28     n = len(nums)
29     res = [1] * n
30     left = 1
31     for i in range(n):
32         res[i] = left; left *= nums[i]
33     right = 1
34     for i in range(n-1, -1, -1):
35         res[i] *= right; right *= nums[i]
36     return res
37
38 #           2D Prefix Sum

39 def build_2d_prefix(mat):
40     m, n = len(mat), len(mat[0])
41     P = [[0]*(n+1) for _ in range(m+1)]
42     for i in range(1, m+1):

```

```

42         for j in range(1, n+1):
43             P[i][j] = mat[i-1][j-1] + P[i-1][j] + P[i][j-1] - P[i
-1][j-1]
44         return P
45
46 def rect_sum(P, r1, c1, r2, c2): # O(1) query
47     return P[r2+1][c2+1] - P[r1][c2+1] - P[r2+1][c1] + P[r1][c1]

```

3.6 Kadane's Algorithm

```

1  #           Maximum Subarray Sum           O(n)   SC: O(1)

2  def max_subarray(nums):
3      best = curr = nums[0]
4      for x in nums[1:]:
5          curr = max(x, curr + x)
6          best = max(best, curr)
7      return best
8
9  #           With Indices

10 def max_subarray_indices(nums):
11     best = curr = nums[0]
12     start = end = temp_start = 0
13     for i in range(1, len(nums)):
14         if nums[i] > curr + nums[i]:
15             curr = nums[i]; temp_start = i
16         else:
17             curr += nums[i]
18         if curr > best:
19             best = curr; start = temp_start; end = i
20     return best, start, end
21
22 #           Maximum Circular Subarray Sum           O(n)

23 def max_circular(nums):
24     def kadane_max(a):
25         cur = res = a[0]
26         for x in a[1:]: cur = max(x, cur+x); res = max(res, cur)
27         return res
28     def kadane_min(a):
29         cur = res = a[0]
30         for x in a[1:]: cur = min(x, cur+x); res = min(res, cur)
31         return res
32     max_straight = kadane_max(nums)
33     circular = sum(nums) - kadane_min(nums)
34     return max(max_straight, circular) if circular != 0 else
max_straight

```

3.7 Other Important Array Algorithms

```

1  #           Boyer-Moore Majority Vote            $O(n)$    SC:  $O(1)$ 

2  def majority_element(nums):           # element appearing > n/2 times
3      candidate = count = 0
4      for x in nums:
5          if count == 0: candidate = x
6          count += 1 if x == candidate else -1
7      return candidate
8
9  #           Dutch National Flag (3-way partition)            $O(n)$ 

10 def sort_colors(nums):                 # sort 0s, 1s, 2s in one pass
11     lo = mid = 0; hi = len(nums) - 1
12     while mid <= hi:
13         if nums[mid] == 0: nums[lo], nums[mid] = nums[mid], nums[
14 lo]; lo += 1; mid += 1
15         elif nums[mid] == 1: mid += 1
16         else: nums[mid], nums[hi] = nums[hi], nums[
17 mid]; hi -= 1
18
19 #           Rotate Array Right by k            $O(n)$    SC:  $O(1)$ 

20 def rotate(nums, k):
21     n = len(nums); k %= n
22     def rev(l, r):
23         while l < r: nums[l], nums[r] = nums[r], nums[l]; l += 1; r
24         -= 1
25     rev(0, n-1); rev(0, k-1); rev(k, n-1)
26
27 #           Next Permutation            $O(n)$ 

28 def next_permutation(nums):
29     n = len(nums); i = n - 2
30     while i >= 0 and nums[i] >= nums[i+1]: i -= 1
31     if i >= 0:
32         j = n - 1
33         while nums[j] <= nums[i]: j -= 1
34         nums[i], nums[j] = nums[j], nums[i]
35     lo, hi = i+1, n-1
36     while lo < hi: nums[lo], nums[hi] = nums[hi], nums[lo]; lo += 1;
37     hi -= 1
38
39 #           Find All Duplicates (using sign trick)            $O(n)$    SC:  $O$ 
40 (1)
41 def find_duplicates(nums):                 # elements in [1, n]
42     result = []
43     for x in nums:
44         idx = abs(x) - 1
45         if nums[idx] < 0: result.append(abs(x))
46         else: nums[idx] = -nums[idx]
47     return result
48
49 #           Longest Consecutive Sequence            $O(n)$ 

50 def longest_consecutive(nums):
51     num_set = set(nums)

```

```
47     best = 0
48     for x in num_set:
49         if x - 1 not in num_set:    # start of a sequence
50             cur = x; length = 1
51             while cur + 1 in num_set: cur += 1; length += 1
52             best = max(best, length)
53     return best
```

Chapter 4

Linked Lists

4.1 Node Class and Singly Linked List

```
1 class ListNode:
2     def __init__(self, val=0, next=None):
3         self.val = val
4         self.next = next
5
6 class SinglyLinkedList:
7     def __init__(self):
8         self.head = None
9         self.size = 0
10
11     #          Insert at head          O(1)
12
13     def push_front(self, val):
14         self.head = ListNode(val, self.head)
15         self.size += 1
16
17     #          Insert at tail          O(n) or O(1) with tail pointer
18
19     def push_back(self, val):
20         if not self.head:
21             self.head = ListNode(val); self.size += 1; return
22         cur = self.head
23         while cur.next: cur = cur.next
24         cur.next = ListNode(val)
25         self.size += 1
26
27     #          Insert at position k (0-indexed)          O(k)
28
29     def insert(self, k, val):
30         if k == 0: self.push_front(val); return
31         cur = self.head
32         for _ in range(k - 1):
33             if not cur: return
34             cur = cur.next
35         node = ListNode(val, cur.next)
36         cur.next = node
37         self.size += 1
38
39     #          Delete first node with given value          O(n)
40
41     def delete(self, val):
42         dummy = ListNode(0, self.head)
43         cur = dummy
44         while cur.next:
```

```

41         if cur.next.val == val:
42             cur.next = cur.next.next; self.size -= 1; break
43         cur = cur.next
44     self.head = dummy.next
45
46     #           Search           O(n)
47
48     def search(self, val):
49         cur = self.head
50         while cur:
51             if cur.val == val: return True
52             cur = cur.next
53         return False
54
55     #           Reverse in-place           O(n)   SC: O(1)
56
57     def reverse(self):
58         prev = None; cur = self.head
59         while cur:
60             nxt = cur.next
61             cur.next = prev
62             prev = cur; cur = nxt
63         self.head = prev
64
65     #           Print
66
67     def to_list(self):
68         res = []; cur = self.head
69         while cur: res.append(cur.val); cur = cur.next
70     return res

```

4.2 Doubly Linked List

```

1  class DNode:
2      def __init__(self, val=0, prev=None, next=None):
3          self.val = val
4          self.prev = prev
5          self.next = next
6
7  class DoublyLinkedList:
8      def __init__(self):
9          # sentinel head and tail (dummy nodes)
10         self.head = DNode()
11         self.tail = DNode()
12         self.head.next = self.tail
13         self.tail.prev = self.head
14         self.size = 0
15
16     def _insert_after(self, node, val): # internal helper
17         new = DNode(val, node, node.next)
18         node.next.prev = new
19         node.next = new
20         self.size += 1

```

```

21
22     def push_front(self, val): self._insert_after(self.head, val)
23     def push_back(self, val): self._insert_after(self.tail.prev,
24 val)
25
26     def _remove(self, node): # internal helper
27         node.prev.next = node.next
28         node.next.prev = node.prev
29         self.size -= 1
30
31     def pop_front(self):
32         if self.head.next == self.tail: return None
33         val = self.head.next.val
34         self._remove(self.head.next); return val
35
36     def pop_back(self):
37         if self.tail.prev == self.head: return None
38         val = self.tail.prev.val
39         self._remove(self.tail.prev); return val

```

4.3 Common Linked List Algorithms

```

1  # Reverse a Linked List O(n) SC: O(1)
2  def reverse_list(head):
3      prev = None; cur = head
4      while cur:
5          nxt = cur.next; cur.next = prev; prev = cur; cur = nxt
6      return prev
7
8  # Find Middle (slow-fast pointers) O(n) SC: O(1)
9  def find_middle(head):
10     slow = fast = head
11     while fast and fast.next:
12         slow = slow.next; fast = fast.next.next
13     return slow # for even length: returns second middle
14
15 # Floyd's Cycle Detection O(n) SC: O(1)
16 def has_cycle(head):
17     slow = fast = head
18     while fast and fast.next:
19         slow = slow.next; fast = fast.next.next
20         if slow is fast: return True
21     return False
22
23 def find_cycle_start(head): # start of cycle or None
24     slow = fast = head
25     while fast and fast.next:
26         slow = slow.next; fast = fast.next.next
27         if slow is fast:
28             # move one pointer to head; advance both at speed 1
29             slow = head
30             while slow is not fast:

```

```

31         slow = slow.next; fast = fast.next
32         return slow
33     return None
34
35     # Merge Two Sorted Lists          O(m+n)   SC: O(1)
36
37 def merge_two_sorted(l1, l2):
38     dummy = cur = ListNode(0)
39     while l1 and l2:
40         if l1.val <= l2.val: cur.next = l1; l1 = l1.next
41         else: cur.next = l2; l2 = l2.next
42         cur = cur.next
43     cur.next = l1 if l1 else l2
44     return dummy.next
45
46     # Remove nth Node from End          O(n)   SC: O(1)
47
48 def remove_nth_from_end(head, n):
49     dummy = ListNode(0, head)
50     fast = slow = dummy
51     for _ in range(n + 1): fast = fast.next
52     while fast:
53         fast = fast.next; slow = slow.next
54     slow.next = slow.next.next
55     return dummy.next
56
57     # Reverse Nodes in k-Groups          O(n)   SC: O(k) call stack
58
59 def reverse_k_group(head, k):
60     def reverse(node, k):
61         prev = None; cur = node; count = 0
62         while cur and count < k:
63             nxt = cur.next; cur.next = prev; prev = cur
64             cur = nxt; count += 1
65         return prev, node, cur # (new_head, old_head/new_tail,
66                                next_group_head)
67
68     def count_nodes(node):
69         c = 0
70         while node: c += 1; node = node.next
71         return c
72
73     if count_nodes(head) < k: return head
74     new_head, tail, nxt = reverse(head, k)
75     tail.next = reverse_k_group(nxt, k)
76     return new_head
77
78     # Intersection of Two Linked Lists   O(m+n)   SC: O(1)
79
80 def get_intersection(headA, headB):
81     a, b = headA, headB
82     while a is not b:
83         a = a.next if a else headB
84         b = b.next if b else headA
85     return a # None if no intersection

```

```

82 #           Add Two Numbers (digits as linked lists)           O(max(m,n))

83 def add_two_numbers(l1, l2):
84     dummy = cur = ListNode(0)
85     carry = 0
86     while l1 or l2 or carry:
87         v1 = l1.val if l1 else 0
88         v2 = l2.val if l2 else 0
89         total = v1 + v2 + carry
90         carry = total // 10
91         cur.next = ListNode(total % 10)
92         cur = cur.next
93         if l1: l1 = l1.next
94         if l2: l2 = l2.next
95     return dummy.next
96
97 #           Flatten a Multilevel Doubly Linked List

98 def flatten(head):
99     cur = head
100    while cur:
101        if cur.child:
102            nxt = cur.next
103            cur.next = cur.child
104            cur.child.prev = cur
105            cur.child = None
106            tail = cur.next
107            while tail.next: tail = tail.next
108            tail.next = nxt
109            if nxt: nxt.prev = tail
110        cur = cur.next
111    return head
112
113 #           Sort Linked List (Merge Sort)           O(n log n)   SC: O(log
n)

114 def sort_list(head):
115     if not head or not head.next: return head
116     mid = find_middle(head)
117     second = mid.next; mid.next = None
118     left = sort_list(head)
119     right = sort_list(second)
120     return merge_two_sorted(left, right)
121
122 #           LRU Cache using OrderedDict           all ops O(1)

123 from collections import OrderedDict
124 class LRUCache:
125     def __init__(self, capacity):
126         self.cap = capacity
127         self.cache = OrderedDict() # key -> value, MRU at end
128
129     def get(self, key):
130         if key not in self.cache: return -1
131         self.cache.move_to_end(key)
132         return self.cache[key]
133
134     def put(self, key, value):

```

```
135     if key in self.cache: self.cache.move_to_end(key)
136     self.cache[key] = value
137     if len(self.cache) > self.cap:
138         self.cache.popitem(last=False)      # remove LRU (front)
```

Chapter 5

Stacks

5.1 Stack Implementations

```
1  #           Using list (most common)

2  stack = []
3  stack.append(x)           # push  O(1)
4  stack.pop()              # pop   O(1)  -- raises IndexError if empty
5  stack[-1]                # peek  O(1)
6  len(stack) == 0          # is_empty

7
8  #           Using deque (slightly faster for large data)

9  from collections import deque
10 stack = deque()
11 stack.append(x)           # push
12 stack.pop()              # pop
13 stack[-1]                # peek

14
15 #           Stack class wrapper

16 class Stack:
17     def __init__(self):         self._data = []
18     def push(self, x):         self._data.append(x)
19     def pop(self):             return self._data.pop()
20     def peek(self):           return self._data[-1]
21     def is_empty(self):       return len(self._data) == 0
22     def __len__(self):        return len(self._data)
```

5.2 Classic Stack Problems

```
1  #           Valid Parentheses           O(n)   SC: O(n)

2  def is_valid(s):
3      match = {'(': ')', ')': '(', '[': ']', ']': '['}
4      stack = []
5      for ch in s:
6          if ch in '([':
7              stack.append(ch)
8          else:
9              if not stack or stack[-1] != match[ch]:
10                 return False
11                 stack.pop()
```

```

12     return len(stack) == 0
13
14 #         Min Stack (push, pop, peek, getMin all O(1))
15
16 class MinStack:
17     def __init__(self):
18         self.stack = []
19         self.min_stack = []
20
21     def push(self, val):
22         self.stack.append(val)
23         if not self.min_stack or val <= self.min_stack[-1]:
24             self.min_stack.append(val)
25
26     def pop(self):
27         val = self.stack.pop()
28         if val == self.min_stack[-1]:
29             self.min_stack.pop()
30
31     def top(self):         return self.stack[-1]
32     def getMin(self):      return self.min_stack[-1]
33
34 #         Infix to Postfix (Shunting-Yard)           O(n)
35
36 def infix_to_postfix(expr):
37     prec = {'+':1, '-':1, '*':2, '/':2, '^':3}
38     assoc = {'^': 'R'} # all others left-associative
39     output = []; stack = []
40     for token in expr.split():
41         if token.lstrip('-').isdigit():
42             output.append(token)
43         elif token == '(':
44             stack.append(token)
45         elif token == ')':
46             while stack and stack[-1] != '(':
47                 output.append(stack.pop())
48             stack.pop()
49         else: # operator
50             while (stack and stack[-1] != '(' and
51                   stack[-1] in prec and
52                   (prec[stack[-1]] > prec[token] or
53                    (prec[stack[-1]] == prec[token] and assoc.get(
54 token) != 'R'))):
55                 output.append(stack.pop())
56             stack.append(token)
57     while stack: output.append(stack.pop())
58     return " ".join(output)
59
60 #         Evaluate Postfix           O(n)
61
62 def eval_postfix(expr):
63     stack = []
64     for token in expr.split():
65         if token.lstrip('-').isdigit():
66             stack.append(int(token))
67         else:

```

```

64         b, a = stack.pop(), stack.pop()
65         if token == '+': stack.append(a + b)
66         elif token == '-': stack.append(a - b)
67         elif token == '*': stack.append(a * b)
68         elif token == '/': stack.append(int(a / b))
69     return stack[0]

```

5.3 Monotonic Stack

A monotonic stack maintains elements in either increasing or decreasing order. It solves **Next Greater/Smaller Element**-style problems in $O(n)$.

```

1  #           Next Greater Element (to the right)           O(n)

2  def next_greater(arr):
3      n = len(arr)
4      result = [-1] * n
5      stack = [] # stores indices, values decrease top-to-bottom
6      for i, x in enumerate(arr):
7          while stack and arr[stack[-1]] < x:
8              idx = stack.pop()
9              result[idx] = x
10             stack.append(i)
11     return result

12

13 #           Previous Smaller Element           O(n)

14 def prev_smaller(arr):
15     n = len(arr)
16     result = [-1] * n
17     stack = []
18     for i, x in enumerate(arr):
19         while stack and arr[stack[-1]] >= x:
20             stack.pop()
21         result[i] = arr[stack[-1]] if stack else -1
22         stack.append(i)
23     return result

24

25 #           Stock Span Problem           O(n)

26 def stock_span(prices):
27     stack = [] # stores (price, span)
28     result = []
29     for p in prices:
30         span = 1
31         while stack and stack[-1][0] <= p:
32             span += stack.pop()[1]
33         stack.append((p, span))
34         result.append(span)
35     return result

36

37 #           Largest Rectangle in Histogram           O(n)   SC: O(n)

```

```

38 def largest_rectangle(heights):
39     heights = heights + [0]    # sentinel
40     stack = [-1]               # stack of indices
41     max_area = 0
42     for i, h in enumerate(heights):
43         while stack[-1] != -1 and heights[stack[-1]] >= h:
44             height = heights[stack.pop()]
45             width = i - stack[-1] - 1
46             max_area = max(max_area, height * width)
47         stack.append(i)
48     return max_area
49
50 #           Maximal Rectangle (in binary matrix)           O(m*n)
51
52 def maximal_rectangle(matrix):
53     if not matrix: return 0
54     n = len(matrix[0])
55     heights = [0] * n
56     max_area = 0
57     for row in matrix:
58         for j in range(n):
59             heights[j] = heights[j] + 1 if row[j] == '1' else 0
60         max_area = max(max_area, largest_rectangle(heights))
61     return max_area
62
63 #           Daily Temperatures (Next Warmer Day)           O(n)
64
65 def daily_temperatures(T):
66     n = len(T)
67     result = [0] * n
68     stack = []
69     for i, t in enumerate(T):
70         while stack and T[stack[-1]] < t:
71             idx = stack.pop()
72             result[idx] = i - idx
73         stack.append(i)
74     return result
75
76 #           Remove k Digits to Get Smallest Number         O(n)
77
78 def remove_k_digits(num, k):
79     stack = []
80     for d in num:
81         while k and stack and stack[-1] > d:
82             stack.pop(); k -= 1
83         stack.append(d)
84     # Remove remaining k digits from the end
85     stack = stack[:len(stack)-k]
86     # Remove leading zeros
87     result = "".join(stack).lstrip('0')
88     return result if result else "0"
89
90 #           Score of Parentheses           O(n)   SC: O(n)
91
92 def score_of_parentheses(s):
93     stack = [0]
94     for ch in s:

```

```

91         if ch == '(':
92             stack.append(0)
93         else:
94             v = stack.pop()
95             stack[-1] += max(2 * v, 1)
96     return stack[0]
97
98     # Decode String (e.g. "3[a2[c]]" -> "accaccacc")      O(n)
99
100 def decode_string(s):
101     stack = []
102     cur_str = ""
103     cur_num = 0
104     for ch in s:
105         if ch.isdigit():
106             cur_num = cur_num * 10 + int(ch)
107         elif ch == '[':
108             stack.append((cur_str, cur_num))
109             cur_str = ""; cur_num = 0
110         elif ch == ']':
111             prev_str, num = stack.pop()
112             cur_str = prev_str + cur_str * num
113         else:
114             cur_str += ch
115     return cur_str

```

Stack Complexities

Operation	Time	Space
Push / Pop / Peek	$O(1)$	—
Valid Parentheses	$O(n)$	$O(n)$
Next Greater Element	$O(n)$	$O(n)$
Largest Rectangle	$O(n)$	$O(n)$

Queues and Deques

6.1 Queue

```
1 from collections import deque
2
3 #           Queue using deque           all ops O(1)
4
5 queue = deque()
6 queue.append(x)           # enqueue (rear)
7 queue.popleft()          # dequeue (front)
8 queue[0]                  # front element (peek)
9 len(queue) == 0          # is_empty
10
11 #           queue.Queue (thread-safe)
12
13 import queue
14 q = queue.Queue(maxsize=10) # bounded queue
15 q.put(x)                   # blocks if full
16 q.get()                    # blocks if empty
17 q.empty()
18 q.full()
19
20 #           Circular Queue implementation
21
22 class CircularQueue:
23     def __init__(self, k):
24         self.k = k
25         self.q = [0] * k
26         self.head = self.tail = -1
27         self.size = 0
28
29     def enqueue(self, val):
30         if self.is_full(): return False
31         self.tail = (self.tail + 1) % self.k
32         self.q[self.tail] = val
33         if self.head == -1: self.head = 0
34         self.size += 1; return True
35
36     def dequeue(self):
37         if self.is_empty(): return False
38         if self.head == self.tail: self.head = self.tail = -1
39         else: self.head = (self.head + 1) % self.k
40         self.size -= 1; return True
```

```

39     def front(self):         return self.q[self.head] if not self.
is_empty() else -1
40     def rear(self):         return self.q[self.tail] if not self.
is_empty() else -1
41     def is_empty(self):     return self.size == 0
42     def is_full(self):      return self.size == self.k

```

6.2 Priority Queue

```

1  import heapq
2
3  #           Min-heap (default)
4
5  h = []
6  heapq.heappush(h, 3)
7  heapq.heappush(h, 1)
8  heapq.heappush(h, 2)
9  print(heapq.heappop(h))    # 1
10
11 #           Max-heap (negate values)
12
13 h = []
14 for x in [3, 1, 4, 1, 5]:
15     heapq.heappush(h, -x)
16 print(-heapq.heappop(h))    # 5
17
18 #           Heap with (priority, value) tuples
19
20 tasks = []
21 heapq.heappush(tasks, (2, "low priority"))
22 heapq.heappush(tasks, (0, "urgent"))
23 priority, task = heapq.heappop(tasks)    # (0, "urgent")
24
25 #           Build heap from list           O(n)
26
27 arr = [3, 1, 4, 1, 5, 9, 2, 6]
28 heapq.heapify(arr)           # in-place min-heap
29
30 #           K-th Largest Element           O(n log k)
31
32 def kth_largest(nums, k):
33     h = []
34     for x in nums:
35         heapq.heappush(h, x)
36         if len(h) > k: heapq.heappop(h)
37     return h[0]
38
39 #           Merge K Sorted Lists           O(n log k)

```

```

35 def merge_k_sorted(lists):
36     h = []
37     for i, lst in enumerate(lists):
38         if lst: heapq.heappush(h, (lst[0], i, 0))
39     result = []
40     while h:
41         val, i, j = heapq.heappop(h)
42         result.append(val)
43         if j + 1 < len(lists[i]):
44             heapq.heappush(h, (lists[i][j+1], i, j+1))
45     return result
46
47 #         Median of Data Stream
48
49 class MedianFinder:
50     def __init__(self):
51         self.lo = []    # max-heap (negated) for lower half
52         self.hi = []    # min-heap for upper half
53
54     def add_num(self, num):
55         heapq.heappush(self.lo, -num)
56         # Balance: lo's max must <= hi's min
57         if self.hi and -self.lo[0] > self.hi[0]:
58             heapq.heappush(self.hi, -heapq.heappop(self.lo))
59         # Keep sizes equal or lo has one extra
60         if len(self.lo) > len(self.hi) + 1:
61             heapq.heappush(self.hi, -heapq.heappop(self.lo))
62         elif len(self.hi) > len(self.lo):
63             heapq.heappush(self.lo, -heapq.heappop(self.hi))
64
65     def find_median(self):
66         if len(self.lo) > len(self.hi): return float(-self.lo[0])
67         return (-self.lo[0] + self.hi[0]) / 2.0
68
69 #         Task Scheduler         O(n log n)
70
71 from collections import Counter
72 import heapq
73 def least_interval(tasks, n):
74     freq = Counter(tasks)
75     max_heap = [-f for f in freq.values()]
76     heapq.heapify(max_heap)
77     time = 0
78     queue = []    # (available_time, -count)
79     while max_heap or queue:
80         if queue and queue[0][0] <= time:
81             _, count = queue.pop(0)
82             heapq.heappush(max_heap, count)
83         if max_heap:
84             count = heapq.heappop(max_heap) + 1    # +1 since negated
85             if count < 0:
86                 queue.append((time + n + 1, count))
87         time += 1
88     return time

```

Part III

Non-Linear Data Structures

Trees

7.1 Binary Tree — Traversals

```

1 class TreeNode:
2     def __init__(self, val=0, left=None, right=None):
3         self.val = val
4         self.left = left
5         self.right = right
6
7     #         Recursive Traversals
8
9     def inorder(root):          # left, root, right
10        if not root: return []
11        return inorder(root.left) + [root.val] + inorder(root.right)
12
13    def preorder(root):         # root, left, right
14        if not root: return []
15        return [root.val] + preorder(root.left) + preorder(root.right)
16
17    def postorder(root):        # left, right, root
18        if not root: return []
19        return postorder(root.left) + postorder(root.right) + [root.val]
20
21    #         Iterative Inorder          O(n)   SC: O(h)
22
23    def inorder_iter(root):
24        stack = []; result = []; cur = root
25        while cur or stack:
26            while cur: stack.append(cur); cur = cur.left
27            cur = stack.pop()
28            result.append(cur.val)
29            cur = cur.right
30        return result
31
32    #         Iterative Preorder          O(n)
33
34    def preorder_iter(root):
35        if not root: return []
36        stack = [root]; result = []
37        while stack:
38            node = stack.pop()
39            result.append(node.val)
40            if node.right: stack.append(node.right)
41            if node.left: stack.append(node.left)
42        return result

```

```

40
41 #           Iterative Postorder           O(n)

42 def postorder_iter(root):
43     if not root: return []
44     stack = [root]; result = []
45     while stack:
46         node = stack.pop()
47         result.append(node.val)
48         if node.left: stack.append(node.left)
49         if node.right: stack.append(node.right)
50     return result[::-1]
51
52 #           Level Order (BFS)           O(n)   SC: O(w) w=max width

53 from collections import deque
54 def level_order(root):
55     if not root: return []
56     queue = deque([root]); result = []
57     while queue:
58         level = []
59         for _ in range(len(queue)):
60             node = queue.popleft()
61             level.append(node.val)
62             if node.left: queue.append(node.left)
63             if node.right: queue.append(node.right)
64         result.append(level)
65     return result
66
67 #           Morris Inorder (O(1) space, no stack/recursion)           O(n)

68 def morris_inorder(root):
69     result = []; cur = root
70     while cur:
71         if not cur.left:
72             result.append(cur.val); cur = cur.right
73         else:
74             prev = cur.left
75             while prev.right and prev.right is not cur:
76                 prev = prev.right
77             if not prev.right:
78                 prev.right = cur; cur = cur.left
79             else:
80                 prev.right = None; result.append(cur.val); cur = cur
81                 .right
82     return result

```

7.2 Binary Tree Properties

```

1 #           Height           O(n)

2 def height(root):
3     if not root: return 0

```

```

4     return 1 + max(height(root.left), height(root.right))
5
6     #           Diameter (longest path between any two nodes)           O(n)
7
8     def diameter(root):
9         max_d = [0]
10        def dfs(node):
11            if not node: return 0
12            l = dfs(node.left); r = dfs(node.right)
13            max_d[0] = max(max_d[0], l + r)
14            return 1 + max(l, r)
15        dfs(root); return max_d[0]
16
17    #           Check if Balanced           O(n)
18
19    def is_balanced(root):
20        def check(node):
21            if not node: return 0
22            l = check(node.left); if l == -1: return -1
23            r = check(node.right); if r == -1: return -1
24            return -1 if abs(l - r) > 1 else 1 + max(l, r)
25        return check(root) != -1
26
27    #           Count Nodes           O(log^2 n) for complete tree
28
29    def count_nodes(root):
30        if not root: return 0
31        lh = rh = 0
32        l = r = root
33        while l: lh += 1; l = l.left
34        while r: rh += 1; r = r.right
35        if lh == rh: return (1 << lh) - 1 # perfect tree
36        return 1 + count_nodes(root.left) + count_nodes(root.right)
37
38    #           Lowest Common Ancestor           O(n)
39
40    def lca(root, p, q):
41        if not root or root is p or root is q: return root
42        left = lca(root.left, p, q)
43        right = lca(root.right, p, q)
44        if left and right: return root
45        return left if left else right
46
47    #           Maximum Path Sum           O(n)
48
49    def max_path_sum(root):
50        best = [float('-inf')]
51        def dfs(node):
52            if not node: return 0
53            l = max(0, dfs(node.left))
54            r = max(0, dfs(node.right))
55            best[0] = max(best[0], node.val + l + r)
56            return node.val + max(l, r)
57        dfs(root); return best[0]

```

```

53
54 #           Serialize / Deserialize           O(n)

55 def serialize(root):
56     if not root: return "null"
57     return str(root.val) + "," + serialize(root.left) + "," +
serialize(root.right)
58
59 def deserialize(data):
60     vals = iter(data.split(","))
61     def build():
62         v = next(vals)
63         if v == "null": return None
64         node = TreeNode(int(v))
65         node.left = build()
66         node.right = build()
67         return node
68     return build()
69
70 #           Check if Two Trees are Identical           O(n)

71 def is_same(p, q):
72     if not p and not q: return True
73     if not p or not q or p.val != q.val: return False
74     return is_same(p.left, q.left) and is_same(p.right, q.right)
75
76 #           Invert (Mirror) a Binary Tree           O(n)

77 def invert_tree(root):
78     if not root: return None
79     root.left, root.right = invert_tree(root.right), invert_tree(
root.left)
80     return root
81
82 #           Right Side View (rightmost node at each level)           O(n)

83 def right_side_view(root):
84     result = []; queue = deque([root]) if root else deque()
85     while queue:
86         for i in range(len(queue)):
87             node = queue.popleft()
88             if i == 0: result.append(node.val) # last in level (
reversed BFS)
89             # OR: rightmost = last seen at each level using BFS left
-to-right
90             # Re-do: standard right-side-view
91     return result
92
93 def right_side_view_v2(root):
94     if not root: return []
95     result = []; queue = deque([root])
96     while queue:
97         for i in range(len(queue)):
98             node = queue.popleft()
99             if i == len(queue): result.append(node.val) # this
logic is wrong

```

```

100         if node.left: queue.append(node.left)
101         if node.right: queue.append(node.right)
102     return result
103
104     # Correct right side view:
105     def right_side_view(root):
106         if not root: return []
107         result = []; queue = deque([root])
108         while queue:
109             level_size = len(queue)
110             for i in range(level_size):
111                 node = queue.popleft()
112                 if i == level_size - 1: result.append(node.val)
113                 if node.left: queue.append(node.left)
114                 if node.right: queue.append(node.right)
115         return result

```

7.3 Binary Search Tree (BST)

```

1 class BST:
2     def __init__(self): self.root = None
3
4     #          Insert          O(h) avg O(log n), worst O(n)
5
6     def insert(self, val):
7         def _insert(node, val):
8             if not node: return TreeNode(val)
9             if val < node.val: node.left = _insert(node.left, val)
10            elif val > node.val: node.right = _insert(node.right,
11            val)
12            return node
13            self.root = _insert(self.root, val)
14
15    #          Search          O(h)
16
17    def search(self, val):
18        cur = self.root
19        while cur:
20            if val == cur.val: return cur
21            elif val < cur.val: cur = cur.left
22            else: cur = cur.right
23        return None
24
25    #          Delete          O(h)
26
27    def delete(self, val):
28        def _delete(node, val):
29            if not node: return None
30            if val < node.val:
31                node.left = _delete(node.left, val)
32            elif val > node.val:
33                node.right = _delete(node.right, val)

```

```

30         else:
31             if not node.left: return node.right
32             if not node.right: return node.left
33             # Find inorder successor (min of right subtree)
34             succ = node.right
35             while succ.left: succ = succ.left
36             node.val = succ.val
37             node.right = _delete(node.right, succ.val)
38         return node
39     self.root = _delete(self.root, val)
40
41     # Floor and Ceil O(h)
42
43     def floor(self, val): # largest value <= val
44         res = None; cur = self.root
45         while cur:
46             if cur.val == val: return val
47             elif cur.val < val: res = cur.val; cur = cur.right
48             else: cur = cur.left
49         return res
50
51     def ceil(self, val): # smallest value >= val
52         res = None; cur = self.root
53         while cur:
54             if cur.val == val: return val
55             elif cur.val > val: res = cur.val; cur = cur.left
56             else: cur = cur.right
57         return res
58
59     # Validate BST O(n) SC: O(h)
60
61     def is_valid_bst(root, lo=float('-inf'), hi=float('inf')):
62         if not root: return True
63         if not (lo < root.val < hi): return False
64         return (is_valid_bst(root.left, lo, root.val) and
65                 is_valid_bst(root.right, root.val, hi))
66
67     # Kth Smallest Element O(k) SC: O(h)
68
69     def kth_smallest(root, k):
70         stack = []; cur = root
71         while True:
72             while cur: stack.append(cur); cur = cur.left
73             cur = stack.pop(); k -= 1
74             if k == 0: return cur.val
75             cur = cur.right
76
77     # Inorder Successor O(h)
78
79     def inorder_successor(root, p):
80         succ = None
81         while root:
82             if p.val < root.val: succ = root; root = root.left
83             else: root = root.right

```

```

80     return succ
81
82 #           Convert Sorted Array to Balanced BST           O(n)
83
84 def sorted_array_to_bst(nums):
85     if not nums: return None
86     mid = len(nums) // 2
87     node = TreeNode(nums[mid])
88     node.left = sorted_array_to_bst(nums[:mid])
89     node.right = sorted_array_to_bst(nums[mid+1:])
90     return node
91
92 #           BST to Sorted Doubly Linked List           O(n)
93
94 def bst_to_dll(root):
95     head = prev = [None, None] # head[0], prev[0]
96     def inorder(node):
97         if not node: return
98         inorder(node.left)
99         if not prev[0]:
100             head[0] = node
101         else:
102             prev[0].right = node
103             node.left = prev[0]
104         prev[0] = node
105         inorder(node.right)
106     inorder(root)
107     return head[0]

```

7.4 AVL Tree

An AVL tree is a self-balancing BST where the heights of left and right subtrees differ by at most 1. Four rotations maintain balance: LL, RR, LR, RL.

```

1  class AVLNode:
2      def __init__(self, val):
3          self.val = val
4          self.left = None
5          self.right = None
6          self.height = 1
7
8  class AVLTree:
9      def _h(self, node): return node.height if node else 0
10     def _bf(self, node): return self._h(node.left) - self._h(node.
11         right)
12     def _update_h(self, node): node.height = 1 + max(self._h(node.
13         left), self._h(node.right))
14
15     def _rotate_right(self, y):
16         x = y.left; T2 = x.right
17         x.right = y; y.left = T2
18         self._update_h(y); self._update_h(x)
19         return x
20
21     def _rotate_left(self, x):

```

```

20     y = x.right; T2 = y.left
21     y.left = x; x.right = T2
22     self._update_h(x); self._update_h(y)
23     return y
24
25     def _rebalance(self, node):
26         self._update_h(node)
27         bf = self._bf(node)
28         # LL
29         if bf > 1 and self._bf(node.left) >= 0:
30             return self._rotate_right(node)
31         # LR
32         if bf > 1 and self._bf(node.left) < 0:
33             node.left = self._rotate_left(node.left)
34             return self._rotate_right(node)
35         # RR
36         if bf < -1 and self._bf(node.right) <= 0:
37             return self._rotate_left(node)
38         # RL
39         if bf < -1 and self._bf(node.right) > 0:
40             node.right = self._rotate_right(node.right)
41             return self._rotate_left(node)
42         return node
43
44     def insert(self, node, val):
45         if not node: return AVLNode(val)
46         if val < node.val: node.left = self.insert(node.left, val
47 )
48         elif val > node.val: node.right = self.insert(node.right,
49 val)
50         else: return node
51         return self._rebalance(node)
52
53     def delete(self, node, val):
54         if not node: return None
55         if val < node.val: node.left = self.delete(node.left, val
56 )
57         elif val > node.val: node.right = self.delete(node.right,
58 val)
59         else:
60             if not node.left: return node.right
61             if not node.right: return node.left
62             succ = node.right
63             while succ.left: succ = succ.left
64             node.val = succ.val
65             node.right = self.delete(node.right, succ.val)
66             return self._rebalance(node)

```

BST / AVL Complexities

Operation	BST Avg	BST Worst	AVL
Search	$O(\log n)$	$O(n)$	$O(\log n)$
Insert	$O(\log n)$	$O(n)$	$O(\log n)$
Delete	$O(\log n)$	$O(n)$	$O(\log n)$
Space	$O(n)$	$O(n)$	$O(n)$

Python Alternative to Balanced BST

Python lacks a built-in balanced BST. Use `sortedcontainers.SortedList` from PyPI for $O(\log n)$ insert/delete/search on a sorted sequence. In competitive programming, use it in place of C++'s `std::set` or `std::multiset`.

```
1 from sortedcontainers import SortedList
2 sl = SortedList()
3 sl.add(3); sl.add(1); sl.add(2)
4 sl.discard(1)
5 sl.bisect_left(2)      # index
```

Heaps and Priority Queues

8.1 Heap Operations via heapq

```

1  import heapq
2
3  # Python provides a MIN-HEAP only.
4  # For max-heap, negate all values.
5
6  #           Basic min-heap
7
8  h = []
9  heapq.heappush(h, 5)
10 heapq.heappush(h, 2)
11 heapq.heappush(h, 8)
12 heapq.heappop(h)           # returns 2 (minimum)
13 h[0]                       # peek minimum without popping
14
15 #           Heapify in O(n)
16
17 arr = [5, 3, 8, 1, 9]
18 heapq.heapify(arr)
19
20 #           Max-heap via negation
21
22 max_h = []
23 for x in [5, 3, 8, 1, 9]:
24     heapq.heappush(max_h, -x)
25 -heapq.heappop(max_h)      # returns 9
26
27 #           heappushpop / heapreplace
28
29 heapq.heappushpop(h, x)    # push x then pop min (more efficient)
30 heapq.heapreplace(h, x)    # pop min then push x (min <= x required)
31
32 #           Custom objects: use (priority, counter, obj)
33
34 import itertools
35 counter = itertools.count() # tie-breaker for equal priorities
36
37 class PQ:
38     def __init__(self):
39         self._h = []
40         self._counter = itertools.count()
41         self._removed = set()

```

```

37
38     def push(self, priority, item):
39         cnt = next(self._counter)
40         heapq.heappush(self._h, (priority, cnt, item))
41
42     def pop(self):
43         while self._h:
44             priority, cnt, item = heapq.heappop(self._h)
45             if cnt not in self._removed:
46                 return priority, item
47             raise KeyError("pop from empty priority queue")
48
49     def remove(self, cnt):
50         self._removed.add(cnt)

```

8.2 Heap Sort

```

1  #           Heap Sort using heapq           O(n log n)   SC: O(n)
2
3  def heap_sort(arr):
4      heapq.heapify(arr)
5      return [heapq.heappop(arr) for _ in range(len(arr))]
6
7  #           In-place Heap Sort               O(n log n)   SC: O(1)
8
9  def heap_sort_inplace(arr):
10     n = len(arr)
11     # Build max-heap
12     for i in range(n // 2 - 1, -1, -1):
13         _sift_down(arr, n, i)
14     # Extract elements one by one
15     for i in range(n-1, 0, -1):
16         arr[0], arr[i] = arr[i], arr[0]
17         _sift_down(arr, i, 0)
18
19     def _sift_down(arr, n, i):
20         largest = i
21         l, r = 2*i + 1, 2*i + 2
22         if l < n and arr[l] > arr[largest]: largest = l
23         if r < n and arr[r] > arr[largest]: largest = r
24         if largest != i:
25             arr[i], arr[largest] = arr[largest], arr[i]
26             _sift_down(arr, n, largest)

```

8.3 Heap Applications

```

1  #           Top K Frequent Elements           O(n log k)
2
3  from collections import Counter
4  import heapq
5
6  def top_k_frequent(nums, k):
7      freq = Counter(nums)
8      return heapq.nlargest(k, freq.keys(), key=freq.get)

```

```

7
8 #           K Closest Points to Origin           O(n log k)

9 def k_closest(points, k):
10     h = []
11     for x, y in points:
12         dist = x*x + y*y
13         heapq.heappush(h, (-dist, x, y))
14         if len(h) > k: heapq.heappop(h)
15     return [[x, y] for _, x, y in h]

16
17 #           Find Kth Smallest in Matrix (sorted rows & cols)           O(k
18     log n)
19 def kth_smallest_matrix(matrix, k):
20     n = len(matrix)
21     h = [(matrix[0][0], 0, 0)]
22     visited = {(0, 0)}
23     for _ in range(k - 1):
24         val, r, c = heapq.heappop(h)
25         for dr, dc in [(0, 1), (1, 0)]:
26             nr, nc = r + dr, c + dc
27             if nr < n and nc < n and (nr, nc) not in visited:
28                 heapq.heappush(h, (matrix[nr][nc], nr, nc))
29                 visited.add((nr, nc))
30     return heapq.heappop(h)[0]

31 #           Minimum Cost to Connect Ropes           O(n log n)

32 def min_cost_ropes(ropes):
33     heapq.heapify(ropes)
34     total = 0
35     while len(ropes) > 1:
36         a = heapq.heappop(ropes)
37         b = heapq.heappop(ropes)
38         cost = a + b
39         total += cost
40         heapq.heappush(ropes, cost)
41     return total

42
43 #           Sliding Window Median           O(n log n)

44 def sliding_window_median(nums, k):
45     from sortedcontainers import SortedList
46     window = SortedList()
47     result = []
48     for i, x in enumerate(nums):
49         window.add(x)
50         if len(window) > k:
51             window.remove(nums[i - k])
52         if len(window) == k:
53             mid = k // 2
54             if k % 2 == 1: result.append(float(window[mid]))
55             else:          result.append((window[mid-1] + window[mid
56 ] / 2.0)
57     return result

```

Chapter 9

Hash Tables and Sets

9.1 Python dict

```
1  # dict (like unordered_map in C++)

2  d = {}
3  d = {'a': 1, 'b': 2}
4  d = dict(zip(keys, values))
5  d = dict.fromkeys(keys, 0)      # all values = 0
6
7  d['key'] = 'val'                # O(1) avg
8  d.get('key', default)          # safe get; no KeyError
9  d.pop('key')                   # remove and return
10 d.pop('key', None)             # safe pop
11 del d['key']
12 'key' in d                      # O(1) avg
13 d.keys(); d.values(); d.items()
14
15 # Merge dicts (Python 3.9+)
16 merged = d1 | d2               # new merged dict
17 d1 |= d2                       # update d1 with d2
18
19 # defaultdict

20 from collections import defaultdict
21 graph = defaultdict(list)      # missing key -> []
22 freq = defaultdict(int)        # missing key -> 0
23 visited = defaultdict(bool)    # missing key -> False
24
25 # Counter

26 from collections import Counter
27 c = Counter([1,2,2,3,3,3])      # {3:3, 2:2, 1:1}
28 c.most_common(2)               # [(3,3), (2,2)]
29 c['z']                         # 0 -- no KeyError
30 c.update([3, 4])              # add more elements
31 c.subtract([1, 2])            # subtract counts
32 c1 + c2                       # sum (keeps positive)
33 c1 - c2                       # difference (drops non-positive)
34 c1 & c2                       # min of each
35 c1 | c2                       # max of each
36 +c                            # remove zero and negative counts
37
38 # OrderedDict
```

```

39 from collections import OrderedDict
40 od = OrderedDict()
41 od.move_to_end('key')           # move to back (LRU cache)
42 od.move_to_end('key', last=False) # move to front
43 od.popitem(last=True)           # pop from back (LIFO)
44 od.popitem(last=False)          # pop from front (FIFO)

```

9.2 Python set and frozenset

```

1  #           set (like unordered_set in C++)

2  s = set()
3  s = {1, 2, 3}
4  s.add(4)           # O(1)
5  s.remove(3)        # KeyError if absent
6  s.discard(3)       # safe remove
7  s.pop()            # remove arbitrary element
8  3 in s              # O(1)

9

10 # Set operations
11 s1 | s2             # union
12 s1 & s2             # intersection
13 s1 - s2             # difference (in s1, not s2)
14 s1 ^ s2             # symmetric difference
15 s1.issubset(s2)
16 s1.issuperset(s2)
17 s1.isdisjoint(s2)

18

19 # frozenset -- immutable, can be used as dict key
20 fs = frozenset([1, 2, 3])

```

9.3 Hash Table Applications

```

1  #           Two Sum           O(n)   SC: O(n)

2  def two_sum(nums, target):
3      seen = {}
4      for i, x in enumerate(nums):
5          complement = target - x
6          if complement in seen: return [seen[complement], i]
7          seen[x] = i
8      return []

9

10 #           Group Anagrams     O(n*k)  SC: O(n*k)

11 def group_anagrams(strs):
12     groups = defaultdict(list)
13     for s in strs:
14         key = tuple(sorted(s))
15         groups[key].append(s)

```

```

16     return list(groups.values())
17
18 #         Longest Consecutive Sequence              $O(n)$ 
19
20 def longest_consecutive(nums):
21     num_set = set(nums)
22     best = 0
23     for x in num_set:
24         if x - 1 not in num_set:
25             cur, length = x, 1
26             while cur + 1 in num_set: cur += 1; length += 1
27             best = max(best, length)
28     return best
29
30 #         Subarray with Zero Sum (exists?)            $O(n)$ 
31
32 def has_zero_sum_subarray(arr):
33     seen = {0}; prefix = 0
34     for x in arr:
35         prefix += x
36         if prefix in seen: return True
37         seen.add(prefix)
38     return False
39
40 #         Implement Custom HashMap
41
42 class MyHashMap:
43     def __init__(self):
44         self.SIZE = 1024
45         self.buckets = [[] for _ in range(self.SIZE)]
46
47     def _bucket(self, key): return key % self.SIZE
48
49     def put(self, key, value):
50         b = self._bucket(key)
51         for i, (k, v) in enumerate(self.buckets[b]):
52             if k == key: self.buckets[b][i] = (key, value); return
53         self.buckets[b].append((key, value))
54
55     def get(self, key):
56         for k, v in self.buckets[self._bucket(key)]:
57             if k == key: return v
58         return -1
59
60     def remove(self, key):
61         b = self._bucket(key)
62         self.buckets[b] = [(k, v) for k, v in self.buckets[b] if k
63 != key]
64
65 #         Isomorphic Strings              $O(n)$ 
66
67 def is_isomorphic(s, t):
68     s2t = {}; t2s = {}
69     for a, b in zip(s, t):
70         if (a in s2t and s2t[a] != b) or (b in t2s and t2s[b] != a):

```

```
66         return False
67         s2t[a] = b; t2s[b] = a
68     return True
69
70 #           Word Pattern           O(n)
71
71 def word_pattern(pattern, s):
72     words = s.split()
73     if len(pattern) != len(words): return False
74     p2w = {}; w2p = {}
75     for p, w in zip(pattern, words):
76         if (p in p2w and p2w[p] != w) or (w in w2p and w2p[w] != p):
77             return False
78         p2w[p] = w; w2p[w] = p
79     return True
```

Tries

10.1 Trie Implementation

```

1  class TrieNode:
2      def __init__(self):
3          self.children = {}      # char -> TrieNode
4          self.is_end = False
5          self.count = 0         # number of words through this node
6
7  class Trie:
8      def __init__(self):
9          self.root = TrieNode()
10
11      #          Insert          O(m) where m = word length
12
13      def insert(self, word):
14          node = self.root
15          for ch in word:
16              if ch not in node.children:
17                  node.children[ch] = TrieNode()
18              node = node.children[ch]
19              node.count += 1
20          node.is_end = True
21
22      #          Search exact word          O(m)
23
24      def search(self, word):
25          node = self.root
26          for ch in word:
27              if ch not in node.children: return False
28              node = node.children[ch]
29          return node.is_end
30
31      #          Prefix exists          O(m)
32
33      def starts_with(self, prefix):
34          node = self.root
35          for ch in prefix:
36              if ch not in node.children: return False
37              node = node.children[ch]
38          return True
39
40      #          Delete          O(m)
41
42      def delete(self, word):

```

```

39     def _del(node, word, i):
40         if i == len(word):
41             if not node.is_end: return False
42             node.is_end = False
43             return len(node.children) == 0
44         ch = word[i]
45         if ch not in node.children: return False
46         should_del = _del(node.children[ch], word, i+1)
47         if should_del:
48             node.children[ch].count -= 1
49             if node.children[ch].count == 0:
50                 del node.children[ch]
51             return not node.is_end and len(node.children) ==
0
52         return False
53     _del(self.root, word, 0)
54
55     #           Count words with prefix           O(m)

56     def count_with_prefix(self, prefix):
57         node = self.root
58         for ch in prefix:
59             if ch not in node.children: return 0
60             node = node.children[ch]
61         return node.count
62
63     #           Longest Common Prefix of all words           O(S)

64     def longest_common_prefix(self):
65         prefix = []
66         node = self.root
67         while len(node.children) == 1 and not node.is_end:
68             ch = next(iter(node.children))
69             prefix.append(ch)
70             node = node.children[ch]
71         return "".join(prefix)
72
73     #           Auto-complete: all words with prefix           O(m + total
)
74     def autocomplete(self, prefix):
75         node = self.root
76         for ch in prefix:
77             if ch not in node.children: return []
78             node = node.children[ch]
79         result = []
80         def dfs(n, curr):
81             if n.is_end: result.append(prefix[:-len(curr)] + curr if
curr else prefix)
82             for ch, child in n.children.items():
83                 dfs(child, curr + ch)
84         def dfs2(n, path):
85             if n.is_end: result.append(path)
86             for ch, child in n.children.items():
87                 dfs2(child, path + ch)
88         dfs2(node, prefix)
89         return result

```

10.2 Trie Applications

```

1  # Word Search II (find all dictionary words in grid)

2  def find_words(board, words):
3      trie = Trie()
4      for w in words: trie.insert(w)
5      m, n = len(board), len(board[0])
6      result = set()
7
8      def dfs(node, r, c, path):
9          ch = board[r][c]
10         if ch not in node.children: return
11         next_node = node.children[ch]
12         path += ch
13         if next_node.is_end: result.add(path)
14         board[r][c] = '#' # mark visited
15         for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]:
16             nr, nc = r+dr, c+dc
17             if 0 <= nr < m and 0 <= nc < n and board[nr][nc] != '#':
18                 dfs(next_node, nr, nc, path)
19         board[r][c] = ch # restore
20
21     for r in range(m):
22         for c in range(n):
23             dfs(trie.root, r, c, "")
24     return list(result)
25
26 # Maximum XOR of Two Numbers using Trie O(n * 32)

27 class BitTrie:
28     def __init__(self): self.root = {}
29
30     def insert(self, num):
31         node = self.root
32         for i in range(31, -1, -1):
33             bit = (num >> i) & 1
34             if bit not in node: node[bit] = {}
35             node = node[bit]
36
37     def max_xor(self, num):
38         node = self.root
39         xor = 0
40         for i in range(31, -1, -1):
41             bit = (num >> i) & 1
42             want = 1 - bit
43             if want in node:
44                 xor |= (1 << i); node = node[want]
45             else:
46                 node = node.get(bit, {})
47         return xor
48
49 def find_maximum_xor(nums):

```

```
50     bt = BitTrie()
51     for x in nums: bt.insert(x)
52     return max(bt.max_xor(x) for x in nums)
```

Part IV

Advanced Data Structures

Disjoint Set Union (DSU / Union-Find)

DSU efficiently answers **connectivity queries**: are two elements in the same set? Two optimisations make every operation nearly $O(1)$: **path compression** and **union by rank/size**.

11.1 Full DSU Implementation

```

1  class DSU:
2      def __init__(self, n):
3          self.parent = list(range(n))
4          self.rank   = [0] * n
5          self.size   = [1] * n
6          self.num_components = n
7
8      # Find with path compression -- O(alpha(n)) amortised
9      def find(self, x):
10         if self.parent[x] != x:
11             self.parent[x] = self.find(self.parent[x])
12         return self.parent[x]
13
14         # Iterative path halving (avoids recursion-limit issues)
15         def find_iter(self, x):
16             while self.parent[x] != x:
17                 self.parent[x] = self.parent[self.parent[x]]    # path
18             halving
19             x = self.parent[x]
20             return x
21
22         # Union by rank -- O(alpha(n)) amortised
23         def union(self, x, y):
24             px, py = self.find(x), self.find(y)
25             if px == py: return False    # already connected
26             if self.rank[px] < self.rank[py]: px, py = py, px
27             self.parent[py] = px
28             self.size[px] += self.size[py]
29             if self.rank[px] == self.rank[py]: self.rank[px] += 1
30             self.num_components -= 1
31             return True
32
33         def connected(self, x, y): return self.find(x) == self.find(y)
34         def get_size(self, x):      return self.size[self.find(x)]
35         def components(self):      return self.num_components

```

11.2 DSU Applications

```

1  from collections import defaultdict
2
3  # Detect Cycle in Undirected Graph -- O(E * alpha(V))
4  def has_cycle(n, edges):
5      dsu = DSU(n)
6      for u, v in edges:
7          if not dsu.union(u, v): return True
8      return False
9
10 # Kruskal's MST -- O(E log E)
11 def kruskal_mst(n, edges):
12     edges.sort(key=lambda e: e[2])
13     dsu = DSU(n)
14     mst_cost = 0; mst_edges = []
15     for u, v, w in edges:
16         if dsu.union(u, v):
17             mst_cost += w; mst_edges.append((u, v, w))
18     return mst_cost, mst_edges
19
20 # Number of Connected Components
21 def count_components(n, edges):
22     dsu = DSU(n)
23     for u, v in edges: dsu.union(u, v)
24     return dsu.components()
25
26 # Redundant Connection -- find edge that creates a cycle
27 def find_redundant_connection(edges):
28     n = len(edges)
29     dsu = DSU(n + 1)
30     for u, v in edges:
31         if not dsu.union(u, v): return [u, v]
32     return []
33
34 # Accounts Merge -- group emails by owner
35 def accounts_merge(accounts):
36     email_id = {}
37     dsu = DSU(len(accounts))
38     for i, acc in enumerate(accounts):
39         for email in acc[1:]:
40             if email in email_id: dsu.union(i, email_id[email])
41             email_id[email] = i
42     groups = defaultdict(set)
43     for email, i in email_id.items():
44         groups[dsu.find(i)].add(email)
45     return [[accounts[i][0]] + sorted(emails) for i, emails in
46             groups.items()]
47
48 # Minimum Number of Operations to Make Network Connected
49 def make_connected(n, connections):
50     if len(connections) < n - 1: return -1    # not enough cables
51     dsu = DSU(n)
52     extra = 0
53     for u, v in connections:
54         if not dsu.union(u, v): extra += 1    # redundant edge
55     components = dsu.components()
56     return components - 1 if extra >= components - 1 else -1

```

Segment Trees

12.1 Range Sum Segment Tree

```

1 class SegTree:
2     def __init__(self, arr):
3         self.n = len(arr)
4         self.tree = [0] * (4 * self.n)
5         if arr: self._build(arr, 0, 0, self.n - 1)
6
7     def _build(self, arr, nd, lo, hi):
8         if lo == hi: self.tree[nd] = arr[lo]; return
9         mid = (lo + hi) // 2
10        self._build(arr, 2*nd+1, lo, mid)
11        self._build(arr, 2*nd+2, mid+1, hi)
12        self.tree[nd] = self.tree[2*nd+1] + self.tree[2*nd+2]
13
14        # Point Update -- O(log n)
15    def update(self, idx, val, nd=0, lo=None, hi=None):
16        if lo is None: lo, hi = 0, self.n - 1
17        if lo == hi: self.tree[nd] = val; return
18        mid = (lo + hi) // 2
19        if idx <= mid: self.update(idx, val, 2*nd+1, lo, mid)
20        else: self.update(idx, val, 2*nd+2, mid+1, hi)
21        self.tree[nd] = self.tree[2*nd+1] + self.tree[2*nd+2]
22
23        # Range Sum Query -- O(log n)
24    def query(self, l, r, nd=0, lo=None, hi=None):
25        if lo is None: lo, hi = 0, self.n - 1
26        if r < lo or hi < l: return 0
27        if l <= lo and hi <= r: return self.tree[nd]
28        mid = (lo + hi) // 2
29        return (self.query(l, r, 2*nd+1, lo, mid) +
30                self.query(l, r, 2*nd+2, mid+1, hi))

```

12.2 Segment Tree with Lazy Propagation

```

1 class LazySegTree:
2     # Range-add update on [l,r]; range sum query
3     def __init__(self, arr):
4         n = self.n = len(arr)
5         self.tree = [0] * (4 * n)
6         self.lazy = [0] * (4 * n)
7         self._build(arr, 0, 0, n - 1)
8
9     def _build(self, arr, nd, lo, hi):
10        if lo == hi: self.tree[nd] = arr[lo]; return

```

```

11     mid = (lo + hi) // 2
12     self._build(arr, 2*nd+1, lo, mid)
13     self._build(arr, 2*nd+2, mid+1, hi)
14     self.tree[nd] = self.tree[2*nd+1] + self.tree[2*nd+2]
15
16     def _push_down(self, nd, lo, hi):
17         if not self.lazy[nd]: return
18         mid = (lo + hi) // 2; d = self.lazy[nd]
19         self.tree[2*nd+1] += d * (mid - lo + 1); self.lazy[2*nd+1]
20 += d
21         self.tree[2*nd+2] += d * (hi - mid); self.lazy[2*nd+2]
22 += d
23         self.lazy[nd] = 0
24
25     # Range add delta to arr[l..r] -- O(log n)
26     def range_add(self, l, r, delta, nd=0, lo=None, hi=None):
27         if lo is None: lo, hi = 0, self.n - 1
28         if r < lo or hi < l: return
29         if l <= lo and hi <= r:
30             self.tree[nd] += delta * (hi - lo + 1); self.lazy[nd] +=
31 delta; return
32         self._push_down(nd, lo, hi)
33         mid = (lo + hi) // 2
34         self.range_add(l, r, delta, 2*nd+1, lo, mid)
35         self.range_add(l, r, delta, 2*nd+2, mid+1, hi)
36         self.tree[nd] = self.tree[2*nd+1] + self.tree[2*nd+2]
37
38     # Range sum query -- O(log n)
39     def query(self, l, r, nd=0, lo=None, hi=None):
40         if lo is None: lo, hi = 0, self.n - 1
41         if r < lo or hi < l: return 0
42         if l <= lo and hi <= r: return self.tree[nd]
43         self._push_down(nd, lo, hi)
44         mid = (lo + hi) // 2
45         return (self.query(l, r, 2*nd+1, lo, mid) + self.query(l, r, 2*nd
46 +2, mid+1, hi))

```

Segment Tree Complexities

Operation	Time	Space
Build	$O(n)$	$O(4n)$
Point update	$O(\log n)$	—
Range query	$O(\log n)$	—
Range update (lazy)	$O(\log n)$	$O(4n)$ for lazy array

Fenwick Tree (Binary Indexed Tree)

```

1  class BIT:
2      # 1-indexed BIT. Supports point update and prefix sum query.
3      def __init__(self, n):
4          self.n = n
5          self.tree = [0] * (n + 1)
6
7      # Add delta to position i -- O(log n)
8      def update(self, i, delta):
9          while i <= self.n:
10             self.tree[i] += delta
11             i += i & (-i)          # add lowest set bit
12
13      # Prefix sum [1..i] -- O(log n)
14      def prefix(self, i):
15          total = 0
16          while i > 0:
17             total += self.tree[i]
18             i -= i & (-i)          # remove lowest set bit
19          return total
20
21      # Range sum [l..r]
22      def range_sum(self, l, r): return self.prefix(r) - self.prefix(l
23          - 1)
24
25      # O(n) build from array (0-indexed input)
26      def build(self, arr):
27          for i, x in enumerate(arr, start=1):
28             self.tree[i] += x
29             j = i + (i & -i)
30             if j <= self.n: self.tree[j] += self.tree[i]
31
32      # Find kth element (1-indexed)
33      def kth(self, k):
34          idx = 0
35          for pw in range(self.n.bit_length(), -1, -1):
36             nxt = idx + (1 << pw)
37             if nxt <= self.n and self.tree[nxt] < k:
38                 idx = nxt; k -= self.tree[nxt]
39          return idx + 1
40
41      # Range Update, Point Query BIT
42      class BIT_RangeUpdate:
43          # Add delta to range [l,r]; query single point i.
44          def __init__(self, n): self.bit = BIT(n)
45
46          def range_update(self, l, r, delta):
47              self.bit.update(l, delta)

```

```

48         self.bit.update(r + 1, -delta)
49
50     def point_query(self, i): return self.bit.prefix(i)
51
52
53     # Range Update, Range Query using two BITs
54     class BIT_RangeRange:
55         # Add delta to [l,r]; query sum of [l,r].
56         def __init__(self, n):
57             self.B1 = BIT(n); self.B2 = BIT(n); self.n = n
58
59         def _prefix(self, i):
60             return self.B1.prefix(i) * i - self.B2.prefix(i)
61
62         def range_update(self, l, r, delta):
63             self.B1.update(l, delta); self.B1.update(r+1, -delta)
64             self.B2.update(l, delta*(l-1)); self.B2.update(r+1, -delta*r
65         )
66
67         def range_query(self, l, r):
68             return self._prefix(r) - self._prefix(l-1)
69
70     # 2D BIT -- O(log m * log n) per operation
71     class BIT2D:
72         def __init__(self, m, n):
73             self.m = m; self.n = n
74             self.tree = [[0]*(n+1) for _ in range(m+1)]
75
76         def update(self, r, c, delta):
77             i = r
78             while i <= self.m:
79                 j = c
80                 while j <= self.n:
81                     self.tree[i][j] += delta; j += j & (-j)
82                 i += i & (-i)
83
84         def prefix(self, r, c):
85             total = 0; i = r
86             while i > 0:
87                 j = c
88                 while j > 0: total += self.tree[i][j]; j -= j & (-j)
89                 i -= i & (-i)
90             return total
91
92         def rect_query(self, r1, c1, r2, c2):
93             return (self.prefix(r2,c2) - self.prefix(r1-1,c2)
94                     - self.prefix(r2,c1-1) + self.prefix(r1-1,c1-1))
95
96
97     # Count Inversions -- O(n log n)
98     def count_inversions(arr):
99         if not arr: return 0
100         compressed = {v: i+1 for i, v in enumerate(sorted(set(arr)))}
101         bit = BIT(len(compressed))
102         inversions = 0
103         for x in arr:

```

```
104         rank = compressed[x]
105         inversions += bit.prefix(len(compressed)) - bit.prefix(rank)
106         bit.update(rank, 1)
107     return inversions
```

Sparse Table (Static RMQ)

A Sparse Table answers **Range Min/Max Queries** in $O(1)$ after $O(n \log n)$ preprocessing. Requires an **idempotent** operation (min, max, gcd); does *not* support updates.

```

1  import math
2
3  class SparseTable:
4      # Range Minimum Query in O(1) after O(n log n) build.
5      def __init__(self, arr):
6          n = len(arr)
7          LOG = max(1, n.bit_length())
8          self.log = [0] * (n + 1)
9          for i in range(2, n + 1):
10             self.log[i] = self.log[i // 2] + 1
11             # st[k][i] = min of arr[i .. i + 2^k - 1]
12             self.st = [arr[:]]
13             for k in range(1, LOG):
14                 prev = self.st[k-1]
15                 half = 1 << (k-1)
16                 self.st.append([min(prev[i], prev[i + half])
17                                for i in range(n - (1 << k) + 1)])
18
19             # O(1) range minimum query
20             def query(self, l, r):
21                 k = self.log[r - l + 1]
22                 return min(self.st[k][l], self.st[k][r - (1 << k) + 1])
23
24
25  class SparseTableMax:
26      # Range Maximum Query in O(1).
27      def __init__(self, arr):
28          n = len(arr); LOG = max(1, n.bit_length())
29          self.log = [0] * (n+1)
30          for i in range(2, n+1): self.log[i] = self.log[i//2]+1
31          self.st = [arr[:]]
32          for k in range(1, LOG):
33              prev = self.st[k-1]; half = 1 << (k-1)
34              self.st.append([max(prev[i], prev[i+half])
35                              for i in range(n-(1<<k)+1)])
36
37          def query(self, l, r):
38              k = self.log[r-l+1]
39              return max(self.st[k][l], self.st[k][r-(1<<k)+1])
40
41
42  # GCD Sparse Table
43  class GCDSparseTable:
44      def __init__(self, arr):
45          import math

```

```

46     n = len(arr); LOG = max(1, n.bit_length())
47     self.log = [0]*(n+1)
48     for i in range(2,n+1): self.log[i]=self.log[i//2]+1
49     self.st = [arr[:]]
50     for k in range(1, LOG):
51         prev = self.st[k-1]; half = 1 << (k-1)
52         self.st.append([math.gcd(prev[i], prev[i+half])
53                         for i in range(n-(1<<k)+1)])
54
55     def query(self, l, r):
56         k = self.log[r-l+1]
57         return math.gcd(self.st[k][l], self.st[k][r-(1<<k)+1])

```

Sparse Table vs Segment Tree vs BIT

Structure	Build	Query	Update
Sparse Table	$O(n \log n)$	$O(1)$	None
Segment Tree	$O(n)$	$O(\log n)$	$O(\log n)$
BIT	$O(n)$	$O(\log n)$	$O(\log n)$

Use Sparse Table for static arrays needing ultra-fast idempotent range queries.

Part V

Graph Algorithms

Graph Representations and Traversals

15.1 Graph Representations

```

1  from collections import defaultdict, deque
2
3  # -- Adjacency List (most common) -- Space:  $O(V + E)$ 
4  graph = defaultdict(list)
5  # Undirected
6  graph[0].append(1); graph[1].append(0)
7  # Directed weighted
8  graph[u].append((v, w))
9
10 # -- Adjacency Matrix -- Space:  $O(V^2)$ 
11 n = 5
12 adj = [[0]*n for _ in range(n)]
13 adj[u][v] = 1          # unweighted
14 adj[u][v] = w          # weighted
15
16 # -- Edge List
17 edges = [(0,1,5), (1,2,3), ...]    # (u, v, weight)
18
19 # -- Building from input
20 def build_graph(n, edge_list, directed=False):
21     g = defaultdict(list)
22     for u, v, *w in edge_list:
23         wt = w[0] if w else 1
24         g[u].append((v, wt))
25         if not directed: g[v].append((u, wt))
26     return g

```

15.2 Depth-First Search (DFS)

```

1  # -- Recursive DFS --  $O(V + E)$ 
2  def dfs_recursive(graph, start):
3      visited = set()
4      order = []
5      def dfs(u):
6          visited.add(u)
7          order.append(u)
8          for v in graph[u]:
9              if v not in visited:
10                 dfs(v)
11     dfs(start)
12     return order
13
14 # -- Iterative DFS (avoids recursion limit on large graphs)

```

```

15 def dfs_iterative(graph, start):
16     visited = set([start])
17     stack = [start]
18     order = []
19     while stack:
20         u = stack.pop()
21         order.append(u)
22         for v in graph[u]:
23             if v not in visited:
24                 visited.add(v)
25                 stack.append(v)
26     return order
27
28 # -- DFS on entire graph (handles disconnected components)
29 def dfs_all(graph, n):
30     visited = set()
31     components = []
32     for src in range(n):
33         if src not in visited:
34             comp = []
35             stack = [src]
36             visited.add(src)
37             while stack:
38                 u = stack.pop()
39                 comp.append(u)
40                 for v in graph[u]:
41                     if v not in visited:
42                         visited.add(v)
43                         stack.append(v)
44             components.append(comp)
45     return components

```

15.3 Breadth-First Search (BFS)

```

1 # -- BFS -- O(V + E), finds shortest path in unweighted graph
2 def bfs(graph, start):
3     visited = set([start])
4     queue = deque([start])
5     order = []
6     while queue:
7         u = queue.popleft()
8         order.append(u)
9         for v in graph[u]:
10             if v not in visited:
11                 visited.add(v)
12                 queue.append(v)
13     return order
14
15 # -- BFS with shortest distance tracking
16 def bfs_distance(graph, start):
17     dist = {start: 0}
18     queue = deque([start])
19     while queue:
20         u = queue.popleft()
21         for v in graph[u]:

```

```

22         if v not in dist:
23             dist[v] = dist[u] + 1
24             queue.append(v)
25     return dist
26
27 # -- BFS with path reconstruction
28 def bfs_path(graph, start, end):
29     parent = {start: None}
30     queue = deque([start])
31     while queue:
32         u = queue.popleft()
33         if u == end:
34             path = []
35             while u is not None:
36                 path.append(u); u = parent[u]
37             return path[::-1]
38         for v in graph[u]:
39             if v not in parent:
40                 parent[v] = u
41                 queue.append(v)
42     return [] # no path
43
44 # -- Multi-source BFS
45 def bfs_multi_source(grid, sources):
46     m, n = len(grid), len(grid[0])
47     dist = [[-1]*n for _ in range(m)]
48     queue = deque()
49     for r, c in sources:
50         dist[r][c] = 0
51         queue.append((r, c))
52     while queue:
53         r, c = queue.popleft()
54         for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]:
55             nr, nc = r+dr, c+dc
56             if 0 <= nr < m and 0 <= nc < n and dist[nr][nc] == -1:
57                 dist[nr][nc] = dist[r][c] + 1
58                 queue.append((nr, nc))
59     return dist

```

15.4 Topological Sort

```

1 # -- Kahn's Algorithm (BFS-based) -- O(V + E)
2 def topo_sort_kahn(graph, n):
3     in_degree = [0] * n
4     for u in range(n):
5         for v in graph[u]: in_degree[v] += 1
6     queue = deque(u for u in range(n) if in_degree[u] == 0)
7     order = []
8     while queue:
9         u = queue.popleft()
10        order.append(u)
11        for v in graph[u]:
12            in_degree[v] -= 1
13            if in_degree[v] == 0: queue.append(v)
14    return order if len(order) == n else [] # empty => cycle

```

```
15
16 # -- DFS-based Topological Sort --  $O(V + E)$ 
17 def topo_sort_dfs(graph, n):
18     WHITE, GRAY, BLACK = 0, 1, 2
19     color = [WHITE] * n
20     order = []
21     has_cycle = [False]
22
23     def dfs(u):
24         color[u] = GRAY
25         for v in graph[u]:
26             if color[v] == GRAY:
27                 has_cycle[0] = True; return
28             if color[v] == WHITE: dfs(v)
29         color[u] = BLACK
30         order.append(u)
31
32     for u in range(n):
33         if color[u] == WHITE: dfs(u)
34     return [] if has_cycle[0] else order[::-1]
35
36 # -- Course Schedule (detect cycle via topo sort) --  $O(V+E)$ 
37 def can_finish(num_courses, prerequisites):
38     graph = defaultdict(list)
39     for a, b in prerequisites: graph[b].append(a)
40     return len(topo_sort_kahn(graph, num_courses)) == num_courses
```

Shortest Path Algorithms

16.1 Dijkstra's Algorithm

```

1  import heapq
2
3  # -- Dijkstra --  $O((V + E) \log V)$  with binary heap
4  def dijkstra(graph, src, n):
5      # graph[u] = [(v, w), ...]
6      dist = [float('inf')] * n
7      dist[src] = 0
8      heap = [(0, src)]      # (dist, node)
9      while heap:
10         d, u = heapq.heappop(heap)
11         if d > dist[u]: continue      # stale entry
12         for v, w in graph[u]:
13             if dist[u] + w < dist[v]:
14                 dist[v] = dist[u] + w
15                 heapq.heappush(heap, (dist[v], v))
16     return dist
17
18 # -- Dijkstra with path reconstruction
19 def dijkstra_path(graph, src, dst, n):
20     dist = [float('inf')] * n
21     parent = [-1] * n
22     dist[src] = 0
23     heap = [(0, src)]
24     while heap:
25         d, u = heapq.heappop(heap)
26         if d > dist[u]: continue
27         if u == dst: break
28         for v, w in graph[u]:
29             if dist[u] + w < dist[v]:
30                 dist[v] = dist[u] + w
31                 parent[v] = u
32                 heapq.heappush(heap, (dist[v], v))
33     if dist[dst] == float('inf'): return [], float('inf')
34     path = []
35     cur = dst
36     while cur != -1: path.append(cur); cur = parent[cur]
37     return path[::-1], dist[dst]
38
39 # -- Dijkstra on grid
40 def dijkstra_grid(grid, start, end):
41     m, n = len(grid), len(grid[0])
42     dist = [[float('inf')] * n for _ in range(m)]
43     sr, sc = start; er, ec = end
44     dist[sr][sc] = 0
45     heap = [(0, sr, sc)]

```

```

46 while heap:
47     d, r, c = heapq.heappop(heap)
48     if d > dist[r][c]: continue
49     if (r, c) == (er, ec): return d
50     for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
51         nr, nc = r+dr, c+dc
52         if 0 <= nr < m and 0 <= nc < n:
53             nd = d + grid[nr][nc]
54             if nd < dist[nr][nc]:
55                 dist[nr][nc] = nd
56                 heapq.heappush(heap, (nd, nr, nc))
57 return dist[er][ec]

```

16.2 Bellman-Ford Algorithm

```

1  # -- Bellman-Ford --  $O(V * E)$ , handles negative weights
2  def bellman_ford(n, edges, src):
3      # edges: [(u, v, w), ...]
4      dist = [float('inf')] * n
5      dist[src] = 0
6      for _ in range(n - 1):          # relax all edges n-1 times
7          updated = False
8          for u, v, w in edges:
9              if dist[u] != float('inf') and dist[u] + w < dist[v]:
10                 dist[v] = dist[u] + w
11                 updated = True
12             if not updated: break    # early termination
13
14     # Detect negative cycle
15     for u, v, w in edges:
16         if dist[u] != float('inf') and dist[u] + w < dist[v]:
17             return None              # negative cycle exists
18     return dist
19
20 # -- SPFA (queue-based Bellman-Ford, faster in practice)
21 def spfa(n, graph, src):
22     dist = [float('inf')] * n
23     in_queue = [False] * n
24     cnt = [0] * n                # count relaxations
25     dist[src] = 0
26     queue = deque([src])
27     in_queue[src] = True
28     while queue:
29         u = queue.popleft()
30         in_queue[u] = False
31         for v, w in graph[u]:
32             if dist[u] + w < dist[v]:
33                 dist[v] = dist[u] + w
34                 cnt[v] += 1
35                 if cnt[v] >= n: return None    # negative cycle
36                 if not in_queue[v]:
37                     queue.append(v); in_queue[v] = True
38     return dist

```

16.3 Floyd-Warshall Algorithm

```

1  # -- Floyd-Warshall --  $O(V^3)$ , all-pairs shortest paths
2  def floyd_warshall(n, edges):
3      INF = float('inf')
4      dist = [[INF]*n for _ in range(n)]
5      for i in range(n): dist[i][i] = 0
6      for u, v, w in edges:
7          dist[u][v] = min(dist[u][v], w)
8          dist[v][u] = min(dist[v][u], w)      # remove for directed
9
10     for k in range(n):
11         for i in range(n):
12             for j in range(n):
13                 if dist[i][k] + dist[k][j] < dist[i][j]:
14                     dist[i][j] = dist[i][k] + dist[k][j]
15
16     # Check for negative cycles ( $dist[i][i] < 0$ )
17     for i in range(n):
18         if dist[i][i] < 0: return None      # negative cycle
19
20     return dist
21
22 # -- Reconstruct path from Floyd-Warshall
23 def floyd_warshall_path(n, edges):
24     INF = float('inf')
25     dist = [[INF]*n for _ in range(n)]
26     next_node = [[None]*n for _ in range(n)]
27     for i in range(n): dist[i][i] = 0
28     for u, v, w in edges:
29         if w < dist[u][v]: dist[u][v] = w; next_node[u][v] = v
30     for k in range(n):
31         for i in range(n):
32             for j in range(n):
33                 if dist[i][k] + dist[k][j] < dist[i][j]:
34                     dist[i][j] = dist[i][k] + dist[k][j]
35                     next_node[i][j] = next_node[i][k]
36     def get_path(u, v):
37         if next_node[u][v] is None: return []
38         path = [u]
39         while u != v: u = next_node[u][v]; path.append(u)
40         return path
41     return dist, get_path

```

Shortest Path Algorithm Comparison

Algorithm	Time	Negative Weights	Neg. Cycles	Use When
BFS	$O(V + E)$	No	No	Unweighted graph
Dijkstra	$O((V + E) \log V)$	No	No	Non-negative weights
Bellman-Ford	$O(VE)$	Yes	Detect	Negative weights
Floyd-Warshall	$O(V^3)$	Yes	Detect	All-pairs, small V

Minimum Spanning Trees

17.1 Kruskal's Algorithm

```

1  # -- Kruskal's MST --  $O(E \log E)$ 
2  def kruskal(n, edges):
3      # edges: [(w, u, v), ...]
4      edges.sort()
5      dsu = DSU(n)
6      mst_weight = 0
7      mst_edges = []
8      for w, u, v in edges:
9          if dsu.union(u, v):
10             mst_weight += w
11             mst_edges.append((u, v, w))
12             if len(mst_edges) == n - 1: break
13     return mst_weight, mst_edges

```

17.2 Prim's Algorithm

```

1  # -- Prim's MST (heap-based) --  $O((V + E) \log V)$ 
2  def prim(graph, n):
3      # graph[u] = [(v, w), ...]
4      in_mst = [False] * n
5      key = [float('inf')] * n
6      parent = [-1] * n
7      key[0] = 0
8      heap = [(0, 0)] # (weight, vertex)
9      mst_cost = 0
10
11     while heap:
12         w, u = heapq.heappop(heap)
13         if in_mst[u]: continue
14         in_mst[u] = True
15         mst_cost += w
16         for v, wt in graph[u]:
17             if not in_mst[v] and wt < key[v]:
18                 key[v] = wt
19                 parent[v] = u
20                 heapq.heappush(heap, (wt, v))
21
22     return mst_cost, [(parent[i], i, key[i]) for i in range(1, n)]

```

Cycle Detection and Graph Properties

18.1 Cycle Detection

```

1  # -- Cycle in Directed Graph (DFS coloring) -- O(V + E)
2  def has_cycle_directed(graph, n):
3      WHITE, GRAY, BLACK = 0, 1, 2
4      color = [WHITE] * n
5
6      def dfs(u):
7          color[u] = GRAY
8          for v in graph[u]:
9              if color[v] == GRAY: return True      # back edge
10             if color[v] == WHITE and dfs(v): return True
11         color[u] = BLACK
12         return False
13
14     return any(color[u] == WHITE and dfs(u) for u in range(n))
15
16 # -- Cycle in Undirected Graph (DFS) -- O(V + E)
17 def has_cycle_undirected(graph, n):
18     visited = [False] * n
19
20     def dfs(u, parent):
21         visited[u] = True
22         for v in graph[u]:
23             if not visited[v]:
24                 if dfs(v, u): return True
25             elif v != parent:
26                 return True      # back edge to non-parent
27         return False
28
29     return any(not visited[u] and dfs(u, -1) for u in range(n))

```

18.2 Strongly Connected Components (SCCs)

```

1  # -- Kosaraju's Algorithm -- O(V + E)
2  def kosaraju(graph, n):
3      # graph: adjacency list (0-indexed)
4      visited = [False] * n
5      order = []
6
7      def dfs1(u):
8          visited[u] = True
9          for v in graph[u]:

```

```

10         if not visited[v]: dfs1(v)
11         order.append(u)
12
13     for u in range(n):
14         if not visited[u]: dfs1(u)
15
16     # Build reverse graph
17     rev = defaultdict(list)
18     for u in range(n):
19         for v in graph[u]: rev[v].append(u)
20
21     visited = [False] * n
22     sccs = []
23
24     def dfs2(u, comp):
25         visited[u] = True; comp.append(u)
26         for v in rev[u]:
27             if not visited[v]: dfs2(v, comp)
28
29     for u in reversed(order):
30         if not visited[u]:
31             comp = []; dfs2(u, comp); sccs.append(comp)
32
33     return sccs
34
35 # -- Tarjan's Algorithm -- O(V + E), single DFS
36 def tarjan_scc(graph, n):
37     index = [0]
38     idx = [-1] * n
39     low = [0] * n
40     on_stack = [False] * n
41     stack = []
42     sccs = []
43
44     def dfs(u):
45         idx[u] = low[u] = index[0]; index[0] += 1
46         stack.append(u); on_stack[u] = True
47
48         for v in graph[u]:
49             if idx[v] == -1:
50                 dfs(v); low[u] = min(low[u], low[v])
51             elif on_stack[v]:
52                 low[u] = min(low[u], idx[v])
53
54         if low[u] == idx[u]: # root of SCC
55             scc = []
56             while True:
57                 w = stack.pop(); on_stack[w] = False
58                 scc.append(w)
59                 if w == u: break
60             sccs.append(scc)
61
62     for u in range(n):
63         if idx[u] == -1: dfs(u)
64
65     return sccs

```

18.3 Bridges and Articulation Points

```

1  # -- Find Bridges (edges whose removal disconnects graph) -- O(V+E)
2  def find_bridges(graph, n):
3      disc = [-1] * n
4      low = [0] * n
5      timer = [0]
6      bridges = []
7
8      def dfs(u, parent):
9          disc[u] = low[u] = timer[0]; timer[0] += 1
10         for v in graph[u]:
11             if v == parent: continue
12             if disc[v] == -1:
13                 dfs(v, u)
14                 low[u] = min(low[u], low[v])
15                 if low[v] > disc[u]:
16                     bridges.append((u, v))
17             else:
18                 low[u] = min(low[u], disc[v])
19
20     for u in range(n):
21         if disc[u] == -1: dfs(u, -1)
22     return bridges
23
24 # -- Find Articulation Points -- O(V + E)
25 def find_articulation_points(graph, n):
26     disc = [-1] * n
27     low = [0] * n
28     timer = [0]
29     is_ap = [False] * n
30
31     def dfs(u, parent):
32         disc[u] = low[u] = timer[0]; timer[0] += 1
33         children = 0
34         for v in graph[u]:
35             if v == parent: continue
36             if disc[v] == -1:
37                 children += 1
38                 dfs(v, u)
39                 low[u] = min(low[u], low[v])
40                 # u is AP if: root with 2+ children, or non-root
41                 with low[v] >= disc[u]
42                 if parent == -1 and children > 1: is_ap[u] = True
43                 if parent != -1 and low[v] >= disc[u]: is_ap[u] =
44                     True
45             else:
46                 low[u] = min(low[u], disc[v])
47
48     for u in range(n):
49         if disc[u] == -1: dfs(u, -1)
50     return [u for u in range(n) if is_ap[u]]

```

18.4 Bipartite Check

```
1  # -- Bipartite Check (2-coloring via BFS) -- O(V + E)
2  def is_bipartite(graph, n):
3      color = [-1] * n
4      for start in range(n):
5          if color[start] != -1: continue
6          queue = deque([start])
7          color[start] = 0
8          while queue:
9              u = queue.popleft()
10             for v in graph[u]:
11                 if color[v] == -1:
12                     color[v] = 1 - color[u]
13                     queue.append(v)
14                 elif color[v] == color[u]:
15                     return False
16         return True
```

Advanced Graph Algorithms

19.1 Flood Fill / Island Problems

```

1  # -- Number of Islands (DFS) -- O(m*n)
2  def num_islands(grid):
3      if not grid: return 0
4      m, n = len(grid), len(grid[0])
5      count = 0
6
7      def dfs(r, c):
8          if r < 0 or r >= m or c < 0 or c >= n or grid[r][c] != '1':
9              return
10             grid[r][c] = '0'      # mark visited (in-place)
11             for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]: dfs(r+dr, c+dc)
12
13     for r in range(m):
14         for c in range(n):
15             if grid[r][c] == '1': count += 1; dfs(r, c)
16     return count
17
18 # -- Surrounded Regions -- O(m*n)
19 def solve_surrounded(board):
20     if not board: return
21     m, n = len(board), len(board[0])
22
23     def dfs(r, c):
24         if r < 0 or r >= m or c < 0 or c >= n or board[r][c] != '0':
25             return
26         board[r][c] = 'S'      # safe
27         for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]: dfs(r+dr, c+dc)
28
29     # Mark border-connected 0's as safe
30     for r in range(m):
31         if board[r][0] == '0': dfs(r, 0)
32         if board[r][n-1] == '0': dfs(r, n-1)
33     for c in range(n):
34         if board[0][c] == '0': dfs(0, c)
35         if board[m-1][c] == '0': dfs(m-1, c)
36
37     for r in range(m):
38         for c in range(n):
39             if board[r][c] == '0': board[r][c] = 'X'
40             elif board[r][c] == 'S': board[r][c] = '0'

```

19.2 0-1 BFS

```

1  # -- 0-1 BFS: edges have weight 0 or 1 --  $O(V + E)$ 
2  # Use deque: append front for weight-0, append back for weight-1
3  def bfs_01(graph, src, n):
4      dist = [float('inf')] * n
5      dist[src] = 0
6      dq = deque([src])
7      while dq:
8          u = dq.popleft()
9          for v, w in graph[u]:
10             if dist[u] + w < dist[v]:
11                 dist[v] = dist[u] + w
12                 if w == 0: dq.appendleft(v)
13                 else:     dq.append(v)
14  return dist

```

19.3 Euler Path and Circuit

```

1  # -- Hierholzer's Algorithm -- Eulerian Circuit --  $O(E)$ 
2  def eulerian_circuit(graph, n):
3      # Precondition: all vertices have even degree
4      # graph is adjacency list with edge counts (multigraph)
5      adj = [list(graph[u]) for u in range(n)]
6      stack = [0]
7      path = []
8      ptr = [0] * n # iterator pointer (avoid  $O(n)$  per removal)
9      # Use iterator approach for efficiency
10     adj_iter = [iter(graph[u]) for u in range(n)]
11     # Simpler version with sets:
12     edge_used = set()
13     adj2 = {u: list(v for v in graph[u]) for u in range(n)}
14     stack2 = [0]; circuit = []
15     while stack2:
16         v = stack2[-1]
17         if adj2[v]:
18             u = adj2[v].pop()
19             adj2[u].remove(v) # undirected: remove back edge
20             stack2.append(u)
21         else:
22             circuit.append(stack2.pop())
23     return circuit[::-1]
24
25 # -- Check Eulerian conditions
26 def has_eulerian_circuit(graph, n):
27     return all(len(graph[u]) % 2 == 0 for u in range(n))
28
29 def has_eulerian_path(graph, n):
30     odd = sum(1 for u in range(n) if len(graph[u]) % 2 == 1)
31     return odd == 0 or odd == 2

```

19.4 Bellman-Ford for Cheapest Flights within K Stops

```

1  # -- Cheapest flights within k stops --  $O(k * E)$ 
2  def find_cheapest_price(n, flights, src, dst, k):

```

```
3     INF = float('inf')
4     dist = [INF] * n
5     dist[src] = 0
6     for _ in range(k + 1):
7         tmp = dist[:]
8         for u, v, w in flights:
9             if dist[u] != INF and dist[u] + w < tmp[v]:
10                 tmp[v] = dist[u] + w
11         dist = tmp
12     return dist[dst] if dist[dst] != INF else -1
```

Part VI

Sorting and Searching

Sorting Algorithms

Python's Built-in Sort

`list.sort()` and `sorted()` use **Timsort** — $O(n \log n)$ worst case, $O(n)$ on nearly-sorted data. Always prefer these in practice. Implement manual sorts only when the algorithm itself is being tested.

20.1 Comparison-Based Sorts

```

1  # =====
2  # BUBBLE SORT --  $O(n^2)$  avg/worst,  $O(n)$  best, Stable
3  # =====
4  def bubble_sort(arr):
5      n = len(arr)
6      for i in range(n):
7          swapped = False
8          for j in range(n - i - 1):
9              if arr[j] > arr[j+1]:
10                 arr[j], arr[j+1] = arr[j+1], arr[j]
11                 swapped = True
12             if not swapped: break # already sorted
13
14  # =====
15  # SELECTION SORT --  $O(n^2)$ , Not stable
16  # =====
17  def selection_sort(arr):
18      n = len(arr)
19      for i in range(n):
20          min_idx = i
21          for j in range(i+1, n):
22              if arr[j] < arr[min_idx]: min_idx = j
23          arr[i], arr[min_idx] = arr[min_idx], arr[i]
24
25  # =====
26  # INSERTION SORT --  $O(n^2)$  avg/worst,  $O(n)$  best, Stable
27  # =====
28  def insertion_sort(arr):
29      for i in range(1, len(arr)):
30          key = arr[i]; j = i - 1
31          while j >= 0 and arr[j] > key:
32              arr[j+1] = arr[j]; j -= 1
33          arr[j+1] = key
34
35  # =====
36  # MERGE SORT --  $O(n \log n)$  all cases, Stable,  $O(n)$  space
37  # =====
38  def merge_sort(arr):

```

```

39     if len(arr) <= 1: return arr
40     mid = len(arr) // 2
41     left = merge_sort(arr[:mid])
42     right = merge_sort(arr[mid:])
43     return _merge(left, right)
44
45 def _merge(left, right):
46     result = []; i = j = 0
47     while i < len(left) and j < len(right):
48         if left[i] <= right[j]: result.append(left[i]); i += 1
49         else: result.append(right[j]); j += 1
50     result.extend(left[i:]); result.extend(right[j:])
51     return result
52
53 # Bottom-up (iterative) merge sort
54 def merge_sort_iterative(arr):
55     n = len(arr)
56     width = 1
57     while width < n:
58         for lo in range(0, n, 2*width):
59             mid = min(lo + width, n)
60             hi = min(lo + 2*width, n)
61             arr[lo:hi] = _merge(arr[lo:mid], arr[mid:hi])
62         width *= 2
63     return arr
64
65 # =====
66 # QUICK SORT --  $O(n \log n)$  avg,  $O(n^2)$  worst, Not stable
67 # =====
68 def quick_sort(arr, lo=0, hi=None):
69     if hi is None: hi = len(arr) - 1
70     if lo >= hi: return
71     p = _partition(arr, lo, hi)
72     quick_sort(arr, lo, p - 1)
73     quick_sort(arr, p + 1, hi)
74
75 def _partition(arr, lo, hi):
76     # Median-of-three pivot
77     mid = (lo + hi) // 2
78     if arr[mid] < arr[lo]: arr[lo], arr[mid] = arr[mid], arr[lo]
79     if arr[hi] < arr[lo]: arr[lo], arr[hi] = arr[hi], arr[lo]
80     if arr[mid] < arr[hi]: arr[mid], arr[hi] = arr[hi], arr[mid]
81     pivot = arr[hi]
82     i = lo - 1
83     for j in range(lo, hi):
84         if arr[j] <= pivot:
85             i += 1; arr[i], arr[j] = arr[j], arr[i]
86     arr[i+1], arr[hi] = arr[hi], arr[i+1]
87     return i + 1
88
89 # Randomised quick sort
90 import random
91 def quick_sort_rand(arr, lo=0, hi=None):
92     if hi is None: hi = len(arr) - 1
93     if lo >= hi: return
94     rand_idx = random.randint(lo, hi)
95     arr[rand_idx], arr[hi] = arr[hi], arr[rand_idx]

```

```

96     p = _partition(arr, lo, hi)
97     quick_sort_rand(arr, lo, p - 1)
98     quick_sort_rand(arr, p + 1, hi)
99
100 # 3-way partition (Dutch National Flag) for many duplicates
101 def quick_sort_3way(arr, lo=0, hi=None):
102     if hi is None: hi = len(arr) - 1
103     if lo >= hi: return
104     lt, gt = lo, hi
105     pivot = arr[lo]; i = lo + 1
106     while i <= gt:
107         if arr[i] < pivot: arr[lt], arr[i] = arr[i], arr[lt]; lt
108         += 1; i += 1
109         elif arr[i] > pivot: arr[i], arr[gt] = arr[gt], arr[i]; gt
110         -= 1
111         else: i += 1
112     quick_sort_3way(arr, lo, lt - 1)
113     quick_sort_3way(arr, gt + 1, hi)
114
115 # =====
116 # HEAP SORT -- O(n log n) all cases, Not stable, O(1) space
117 # =====
118 def heap_sort(arr):
119     n = len(arr)
120     for i in range(n//2 - 1, -1, -1):
121         _sift_down(arr, n, i)
122     for i in range(n-1, 0, -1):
123         arr[0], arr[i] = arr[i], arr[0]
124         _sift_down(arr, i, 0)
125
126 def _sift_down(arr, n, i):
127     largest = i; l = 2*i+1; r = 2*i+2
128     if l < n and arr[l] > arr[largest]: largest = l
129     if r < n and arr[r] > arr[largest]: largest = r
130     if largest != i:
131         arr[i], arr[largest] = arr[largest], arr[i]
132         _sift_down(arr, n, largest)
133
134 # =====
135 # SHELL SORT -- O(n log^2 n) with Ciura gaps, Not stable
136 # =====
137 def shell_sort(arr):
138     gaps = [701, 301, 132, 57, 23, 10, 4, 1] # Ciura gap sequence
139     for gap in gaps:
140         for i in range(gap, len(arr)):
141             temp = arr[i]; j = i
142             while j >= gap and arr[j-gap] > temp:
143                 arr[j] = arr[j-gap]; j -= gap
144             arr[j] = temp

```

20.2 Non-Comparison Sorts

```

1 # =====
2 # COUNTING SORT -- O(n + k), Stable, for integers in [0, k]
3 # =====

```

```

4 def counting_sort(arr, k=None):
5     if not arr: return arr
6     if k is None: k = max(arr)
7     count = [0] * (k + 1)
8     for x in arr: count[x] += 1
9     for i in range(1, k+1): count[i] += count[i-1]    # prefix sums
10    output = [0] * len(arr)
11    for x in reversed(arr):    # reversed for stability
12        count[x] -= 1
13        output[count[x]] = x
14    return output
15
16 # =====
17 # RADIX SORT -- O(d * (n + b)), Stable, d=digits, b=base
18 # =====
19 def radix_sort(arr, base=10):
20     if not arr: return arr
21     max_val = max(arr)
22     exp = 1
23     while max_val // exp > 0:
24         arr = _counting_sort_by_digit(arr, exp, base)
25         exp *= base
26     return arr
27
28 def _counting_sort_by_digit(arr, exp, base):
29     n = len(arr)
30     count = [0] * base
31     output = [0] * n
32     for x in arr: count[(x // exp) % base] += 1
33     for i in range(1, base): count[i] += count[i-1]
34     for x in reversed(arr):
35         idx = (x // exp) % base
36         count[idx] -= 1
37         output[count[idx]] = x
38     return output
39
40 # =====
41 # BUCKET SORT -- O(n + k) avg for uniformly distributed floats
42 # =====
43 def bucket_sort(arr):
44     if not arr: return arr
45     n = len(arr)
46     buckets = [[] for _ in range(n)]
47     min_v, max_v = min(arr), max(arr)
48     span = max_v - min_v + 1e-9
49     for x in arr:
50         idx = int((x - min_v) / span * (n - 1))
51         buckets[idx].append(x)
52     result = []
53     for b in buckets:
54         b.sort()
55         result.extend(b)
56     return result

```

20.3 Python Sort Customisation

```
1  # -- Custom key functions
2  arr.sort(key=lambda x: x[1])           # sort by 2nd element
3  arr.sort(key=lambda x: (-x[0], x[1]))  # desc by first, asc by
    second
4  arr.sort(key=len)                     # sort strings by length
5  arr.sort(key=str.lower)               # case-insensitive
6
7  # -- functools.cmp_to_key (for custom comparator like C++ cmp)
8  from functools import cmp_to_key
9  def my_cmp(a, b):
10     return -1 if str(a)+str(b) > str(b)+str(a) else 1 # largest
    number
11  nums = [3, 30, 34, 5, 9]
12  nums.sort(key=cmp_to_key(my_cmp))
13  print("".join(map(str, nums)))        # "9534330"
14
15  # -- Stable sort: Python's sort is always stable
16  # Equal elements maintain original relative order
17
18  # -- Sort with multiple criteria (tuples)
19  students = [("Alice", 90, 20), ("Bob", 85, 22), ("Carol", 90, 19)]
20  students.sort(key=lambda s: (-s[1], s[2])) # high grade first, then
    younger
```

Searching Algorithms

21.1 Binary Search

```

1  # -- Standard Binary Search -- O(log n)
2  def binary_search(arr, target):
3      lo, hi = 0, len(arr) - 1
4      while lo <= hi:
5          mid = (lo + hi) // 2
6          if arr[mid] == target: return mid
7          elif arr[mid] < target: lo = mid + 1
8          else:                  hi = mid - 1
9      return -1
10
11 # -- First occurrence (leftmost)
12 def lower_bound(arr, target):
13     lo, hi = 0, len(arr)
14     while lo < hi:
15         mid = (lo + hi) // 2
16         if arr[mid] < target: lo = mid + 1
17         else:                hi = mid
18     return lo      # == bisect.bisect_left(arr, target)
19
20 # -- First position > target (rightmost + 1)
21 def upper_bound(arr, target):
22     lo, hi = 0, len(arr)
23     while lo < hi:
24         mid = (lo + hi) // 2
25         if arr[mid] <= target: lo = mid + 1
26         else:                hi = mid
27     return lo      # == bisect.bisect_right(arr, target)
28
29 # -- Binary search on answer ("minimise the maximum" / feasibility)
30 def binary_search_answer(lo, hi, feasible_fn):
31     while lo < hi:
32         mid = (lo + hi) // 2
33         if feasible_fn(mid): hi = mid
34         else:               lo = mid + 1
35     return lo
36
37 # Example: Minimum days to make m bouquets
38 def min_days_bouquets(bloomDay, m, k):
39     if m * k > len(bloomDay): return -1
40     def feasible(day):
41         bouquets = consec = 0
42         for d in bloomDay:
43             if d <= day: consec += 1
44             else:        consec = 0
45             if consec == k: bouquets += 1; consec = 0

```

```

46         return bouquets >= m
47     return binary_search_answer(1, max(bloomDay), feasible)
48
49 # -- Find Peak Element -- O(log n)
50 def find_peak(nums):
51     lo, hi = 0, len(nums) - 1
52     while lo < hi:
53         mid = (lo + hi) // 2
54         if nums[mid] < nums[mid+1]: lo = mid + 1
55         else: hi = mid
56     return lo
57
58 # -- Search in Rotated Sorted Array -- O(log n)
59 def search_rotated(nums, target):
60     lo, hi = 0, len(nums) - 1
61     while lo <= hi:
62         mid = (lo + hi) // 2
63         if nums[mid] == target: return mid
64         if nums[lo] <= nums[mid]: # left half sorted
65             if nums[lo] <= target < nums[mid]: hi = mid - 1
66             else: lo = mid + 1
67         else: # right half sorted
68             if nums[mid] < target <= nums[hi]: lo = mid + 1
69             else: hi = mid - 1
70     return -1
71
72 # -- Find Minimum in Rotated Sorted Array -- O(log n)
73 def find_min_rotated(nums):
74     lo, hi = 0, len(nums) - 1
75     while lo < hi:
76         mid = (lo + hi) // 2
77         if nums[mid] > nums[hi]: lo = mid + 1
78         else: hi = mid
79     return nums[lo]
80
81 # -- Binary search on 2D matrix (sorted rows and cols) -- O(m+n)
82 def search_matrix(matrix, target):
83     m, n = len(matrix), len(matrix[0])
84     r, c = 0, n - 1 # start from top-right corner
85     while r < m and c >= 0:
86         if matrix[r][c] == target: return True
87         elif matrix[r][c] < target: r += 1
88         else: c -= 1
89     return False
90
91 # -- Median of Two Sorted Arrays -- O(log(min(m,n)))
92 def find_median_sorted(nums1, nums2):
93     if len(nums1) > len(nums2): nums1, nums2 = nums2, nums1
94     m, n = len(nums1), len(nums2)
95     lo, hi = 0, m
96     while lo <= hi:
97         i = (lo + hi) // 2
98         j = (m + n + 1) // 2 - i
99         max_l1 = float('-inf') if i == 0 else nums1[i-1]
100         min_r1 = float('inf') if i == m else nums1[i]
101         max_l2 = float('-inf') if j == 0 else nums2[j-1]
102         min_r2 = float('inf') if j == n else nums2[j]

```

```

103     if max_l1 <= min_r2 and max_l2 <= min_r1:
104         if (m + n) % 2 == 1: return float(max(max_l1, max_l2))
105         return (max(max_l1, max_l2) + min(min_r1, min_r2)) / 2.0
106     elif max_l1 > min_r2: hi = i - 1
107     else:
        lo = i + 1

```

21.2 Ternary Search

```

1  # -- Ternary Search (find maximum of unimodal function) -- O(log n)
2  def ternary_search(lo, hi, f, eps=1e-9):
3      while hi - lo > eps:
4          m1 = lo + (hi - lo) / 3
5          m2 = hi - (hi - lo) / 3
6          if f(m1) < f(m2): lo = m1
7          else:
8              hi = m2
9      return (lo + hi) / 2
10
11 # Integer version
12 def ternary_search_int(lo, hi, arr):
13     while hi - lo > 2:
14         m1 = lo + (hi - lo) // 3
15         m2 = hi - (hi - lo) // 3
16         if arr[m1] < arr[m2]: lo = m1
17         else:
18             hi = m2
19     return max(range(lo, hi+1), key=lambda i: arr[i])

```

Sorting Complexity Summary

Algorithm	Best	Average	Worst	Stable
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	No
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	Yes
Radix Sort	$O(dn)$	$O(dn)$	$O(dn)$	Yes
Timsort (Python)	$O(n)$	$O(n \log n)$	$O(n \log n)$	Yes

Part VII

Algorithm Design Paradigms

Dynamic Programming

DP in Python vs C++

Python's `@cache` / `@lru_cache` makes top-down memoisation trivially easy. For bottom-up, use lists. Always call `sys.setrecursionlimit` for deep recursion.

22.1 1-D DP

```

1  import sys
2  from functools import cache
3  sys.setrecursionlimit(300000)
4
5  # -- Fibonacci -- O(n) time, O(n) -> O(1) space
6  @cache
7  def fib(n):
8      if n <= 1: return n
9      return fib(n-1) + fib(n-2)
10
11 # Bottom-up O(1) space
12 def fib_iterative(n):
13     a, b = 0, 1
14     for _ in range(n): a, b = b, a + b
15     return a
16
17 # -- Climbing Stairs -- O(n)
18 def climb_stairs(n):
19     if n <= 2: return n
20     a, b = 1, 2
21     for _ in range(3, n+1): a, b = b, a + b
22     return b
23
24 # -- House Robber -- O(n) time, O(1) space
25 def rob(nums):
26     prev2 = prev1 = 0
27     for x in nums:
28         prev2, prev1 = prev1, max(prev1, prev2 + x)
29     return prev1
30
31 # House Robber II (circular array)
32 def rob2(nums):
33     def rob_linear(arr):
34         p2 = p1 = 0
35         for x in arr: p2, p1 = p1, max(p1, p2 + x)
36         return p1
37     return max(nums[0], rob_linear(nums[1:]), rob_linear(nums[:-1]))
38
39 # -- Coin Change (minimum coins) -- O(n * amount)

```

```

40 def coin_change(coins, amount):
41     dp = [float('inf')] * (amount + 1)
42     dp[0] = 0
43     for c in coins:
44         for a in range(c, amount + 1):
45             dp[a] = min(dp[a], dp[a - c] + 1)
46     return dp[amount] if dp[amount] != float('inf') else -1
47
48 # -- Coin Change II (number of ways) -- O(n * amount)
49 def change(amount, coins):
50     dp = [0] * (amount + 1)
51     dp[0] = 1
52     for c in coins:
53         for a in range(c, amount+1):
54             dp[a] += dp[a - c]
55     return dp[amount]
56
57 # -- Word Break -- O(n^2 * m) where m = avg word length
58 def word_break(s, word_dict):
59     word_set = set(word_dict)
60     n = len(s)
61     dp = [False] * (n + 1)
62     dp[0] = True
63     for i in range(1, n + 1):
64         for j in range(i):
65             if dp[j] and s[j:i] in word_set:
66                 dp[i] = True; break
67     return dp[n]
68
69 # -- Decode Ways -- O(n)
70 def num_decodings(s):
71     n = len(s)
72     dp = [0] * (n + 1)
73     dp[0] = 1
74     dp[1] = 0 if s[0] == '0' else 1
75     for i in range(2, n + 1):
76         one = int(s[i-1:i])
77         two = int(s[i-2:i])
78         if 1 <= one <= 9: dp[i] += dp[i-1]
79         if 10 <= two <= 26: dp[i] += dp[i-2]
80     return dp[n]
81
82 # -- Jump Game II (minimum jumps) -- O(n)
83 def jump(nums):
84     jumps = cur_end = cur_far = 0
85     for i in range(len(nums) - 1):
86         cur_far = max(cur_far, i + nums[i])
87         if i == cur_end:
88             jumps += 1; cur_end = cur_far
89     return jumps

```

22.2 2-D DP

```

1 # -- 0/1 Knapsack -- O(n * W)

```

```

2 def knapsack_01(weights, values, W):
3     n = len(weights)
4     dp = [[0]*(W+1) for _ in range(n+1)]
5     for i in range(1, n+1):
6         for w in range(W+1):
7             dp[i][w] = dp[i-1][w]
8             if weights[i-1] <= w:
9                 dp[i][w] = max(dp[i][w], dp[i-1][w-weights[i-1]] +
values[i-1])
10    return dp[n][W]
11
12 # Space optimised O(W)
13 def knapsack_01_opt(weights, values, W):
14     dp = [0] * (W + 1)
15     for w_i, v_i in zip(weights, values):
16         for w in range(W, w_i - 1, -1): # traverse backward
17             dp[w] = max(dp[w], dp[w - w_i] + v_i)
18     return dp[W]
19
20 # -- Unbounded Knapsack -- O(n * W)
21 def knapsack_unbounded(weights, values, W):
22     dp = [0] * (W + 1)
23     for w in range(1, W + 1):
24         for w_i, v_i in zip(weights, values):
25             if w_i <= w:
26                 dp[w] = max(dp[w], dp[w - w_i] + v_i)
27     return dp[W]
28
29 # -- Longest Common Subsequence -- O(m*n)
30 def lcs(s, t):
31     m, n = len(s), len(t)
32     dp = [[0]*(n+1) for _ in range(m+1)]
33     for i in range(1, m+1):
34         for j in range(1, n+1):
35             if s[i-1] == t[j-1]: dp[i][j] = dp[i-1][j-1] + 1
36             else: dp[i][j] = max(dp[i-1][j], dp[i][
j-1])
37     return dp[m][n]
38
39 # LCS reconstruction
40 def lcs_string(s, t):
41     m, n = len(s), len(t)
42     dp = [[0]*(n+1) for _ in range(m+1)]
43     for i in range(1, m+1):
44         for j in range(1, n+1):
45             if s[i-1] == t[j-1]: dp[i][j] = dp[i-1][j-1] + 1
46             else: dp[i][j] = max(dp[i-1][j], dp[i][
j-1])
47     result = []
48     i, j = m, n
49     while i > 0 and j > 0:
50         if s[i-1] == t[j-1]: result.append(s[i-1]); i -= 1; j -= 1
51         elif dp[i-1][j] > dp[i][j-1]: i -= 1
52         else: j -= 1
53     return "".join(reversed(result))
54
55 # -- Longest Increasing Subsequence -- O(n log n) with patience sort

```

```

56 import bisect
57 def lis(nums):
58     tails = []
59     for x in nums:
60         pos = bisect.bisect_left(tails, x)
61         if pos == len(tails): tails.append(x)
62         else: tails[pos] = x
63     return len(tails)
64
65 # LIS  $O(n^2)$  with reconstruction
66 def lis_with_path(nums):
67     n = len(nums)
68     dp = [1] * n
69     parent = [-1] * n
70     for i in range(1, n):
71         for j in range(i):
72             if nums[j] < nums[i] and dp[j] + 1 > dp[i]:
73                 dp[i] = dp[j] + 1; parent[i] = j
74     end = dp.index(max(dp))
75     path = []
76     while end != -1: path.append(nums[end]); end = parent[end]
77     return max(dp), path[::-1]
78
79 # -- Edit Distance (Levenshtein) --  $O(m*n)$ 
80 def edit_distance(s, t):
81     m, n = len(s), len(t)
82     dp = list(range(n + 1))
83     for i in range(1, m+1):
84         prev = dp[:]
85         dp[0] = i
86         for j in range(1, n+1):
87             if s[i-1] == t[j-1]: dp[j] = prev[j-1]
88             else: dp[j] = 1 + min(prev[j], dp[j-1], prev[j-1])
89     return dp[n]
90
91 # -- Unique Paths --  $O(m*n)$ 
92 def unique_paths(m, n):
93     dp = [1] * n
94     for _ in range(1, m):
95         for j in range(1, n): dp[j] += dp[j-1]
96     return dp[n-1]
97
98 # -- Minimum Path Sum --  $O(m*n)$ 
99 def min_path_sum(grid):
100     m, n = len(grid), len(grid[0])
101     dp = grid[0][:]
102     for j in range(1, n): dp[j] += dp[j-1]
103     for i in range(1, m):
104         dp[0] += grid[i][0]
105         for j in range(1, n):
106             dp[j] = grid[i][j] + min(dp[j], dp[j-1])
107     return dp[n-1]
108
109 # -- Regular Expression Matching --  $O(m*n)$ 
110 def is_match(s, p):
111     m, n = len(s), len(p)
112     dp = [[False]*(n+1) for _ in range(m+1)]

```

```

113 dp[0][0] = True
114 for j in range(1, n+1):
115     if p[j-1] == '*': dp[0][j] = dp[0][j-2]
116 for i in range(1, m+1):
117     for j in range(1, n+1):
118         if p[j-1] in (s[i-1], '.'):
119             dp[i][j] = dp[i-1][j-1]
120         elif p[j-1] == '*':
121             dp[i][j] = dp[i][j-2] # zero occurrences
122             if p[j-2] in (s[i-1], '.'):
123                 dp[i][j] = dp[i][j] or dp[i-1][j]
124 return dp[m][n]

```

22.3 Interval DP

```

1 # -- Matrix Chain Multiplication -- O(n^3)
2 def matrix_chain(dims):
3     n = len(dims) - 1 # number of matrices
4     dp = [[0]*n for _ in range(n)]
5     for length in range(2, n+1):
6         for i in range(n - length + 1):
7             j = i + length - 1
8             dp[i][j] = float('inf')
9             for k in range(i, j):
10                cost = dp[i][k] + dp[k+1][j] + dims[i]*dims[k+1]*
11                dims[j+1]
12                dp[i][j] = min(dp[i][j], cost)
13            return dp[0][n-1]
14
15 # -- Burst Balloons -- O(n^3)
16 def max_coins(nums):
17     nums = [1] + nums + [1]
18     n = len(nums)
19     dp = [[0]*n for _ in range(n)]
20     for length in range(2, n):
21         for lo in range(n - length):
22             hi = lo + length
23             for k in range(lo+1, hi):
24                 dp[lo][hi] = max(dp[lo][hi],
25                 dp[lo][k] + nums[lo]*nums[k]*nums[hi] + dp[k][hi
26             ])
27         return dp[0][n-1]
28
29 # -- Minimum Cost to Cut a Stick -- O(n^3)
30 def min_cost_cut(n, cuts):
31     cuts = [0] + sorted(cuts) + [n]
32     m = len(cuts)
33     dp = [[0]*m for _ in range(m)]
34     for length in range(2, m):
35         for lo in range(m - length):
36             hi = lo + length
37             dp[lo][hi] = min(cuts[hi]-cuts[lo] + dp[lo][k] + dp[k][
38             hi]
39             for k in range(lo+1, hi))
40         return dp[0][m-1]

```

```

38
39 # -- Palindrome Partitioning II (min cuts) -- O(n^2)
40 def min_cut(s):
41     n = len(s)
42     is_pal = [[False]*n for _ in range(n)]
43     for i in range(n-1, -1, -1):
44         for j in range(i, n):
45             if s[i] == s[j] and (j-i <= 2 or is_pal[i+1][j-1]):
46                 is_pal[i][j] = True
47     dp = list(range(-1, n)) # dp[i] = min cuts for s[0..i]
48     for i in range(n):
49         for j in range(i+1):
50             if is_pal[j][i]:
51                 dp[i+1] = min(dp[i+1], dp[j] + 1)
52     return dp[n]

```

22.4 DP on Trees

```

1 # -- Maximum Independent Set on Tree -- O(n)
2 def tree_max_independent_set(adj, n):
3     dp = [[0, 0] for _ in range(n)] # [excl_root, incl_root]
4
5     def dfs(u, parent):
6         dp[u][1] = 1 # include u
7         for v in adj[u]:
8             if v == parent: continue
9             dfs(v, u)
10            dp[u][0] += max(dp[v][0], dp[v][1])
11            dp[u][1] += dp[v][0]
12
13     dfs(0, -1)
14     return max(dp[0][0], dp[0][1])
15
16 # -- Diameter of Tree (longest path) -- O(n)
17 def tree_diameter(adj, n):
18     dist = [0] * n
19     def dfs(u, p):
20         max1 = max2 = 0
21         for v in adj[u]:
22             if v == p: continue
23             dfs(v, u)
24             d = dist[v] + 1
25             if d > max1: max2, max1 = max1, d
26             elif d > max2: max2 = d
27         dist[u] = max1
28         return max1 + max2 # path through u
29     ans = [0]
30     def dfs2(u, p):
31         r = dfs(u, p)
32         ans[0] = max(ans[0], r)
33     dfs2(0, -1)
34     return ans[0]

```

22.5 Digit DP

```
1  # -- Count numbers in [0, n] with digit sum <= S -- O(digits * S)
2  def count_digit_sum(n, S):
3      digits = [int(d) for d in str(n)]
4      from functools import cache
5
6      @cache
7      def dp(pos, current_sum, tight, started):
8          if current_sum > S: return 0
9          if pos == len(digits):
10             return 1 if started else 0
11             limit = digits[pos] if tight else 9
12             result = 0
13             for d in range(0, limit + 1):
14                 result += dp(pos+1,
15                             current_sum + (d if started or d > 0 else
16                             0),
17                             tight and d == limit,
18                             started or d > 0)
19             return result
20     return dp(0, 0, True, False)
```

Greedy Algorithms

```

1  # -- Activity Selection (max non-overlapping intervals) --  $O(n \log n)$ 
2  )
3  def activity_selection(intervals):
4      intervals.sort(key=lambda x: x[1])      # sort by end time
5      count = 0; last_end = float('-inf')
6      for start, end in intervals:
7          if start >= last_end:
8              count += 1; last_end = end
9      return count
10
11 # -- Minimum Number of Platforms (trains) --  $O(n \log n)$ 
12 def min_platforms(arrivals, departures):
13     arrivals.sort(); departures.sort()
14     platforms = max_platforms = 0
15     i = j = 0
16     while i < len(arrivals):
17         if arrivals[i] <= departures[j]:
18             platforms += 1; i += 1
19             max_platforms = max(max_platforms, platforms)
20         else:
21             platforms -= 1; j += 1
22     return max_platforms
23
24 # -- Gas Station (circular tour) --  $O(n)$ 
25 def can_complete_circuit(gas, cost):
26     total = tank = start = 0
27     for i in range(len(gas)):
28         diff = gas[i] - cost[i]
29         total += diff; tank += diff
30         if tank < 0: start = i + 1; tank = 0
31     return start if total >= 0 else -1
32
33 # -- Jump Game (can reach end?) --  $O(n)$ 
34 def can_jump(nums):
35     max_reach = 0
36     for i, x in enumerate(nums):
37         if i > max_reach: return False
38         max_reach = max(max_reach, i + x)
39     return True
40
41 # -- Assign Cookies --  $O(n \log n)$ 
42 def find_content_children(greed, size):
43     greed.sort(); size.sort()
44     child = cookie = 0
45     while child < len(greed) and cookie < len(size):
46         if size[cookie] >= greed[child]: child += 1
47         cookie += 1
48     return child

```

```

48
49 # -- Fractional Knapsack -- O(n log n)
50 def fractional_knapsack(weights, values, W):
51     items = sorted(zip(values, weights), key=lambda x: x[0]/x[1],
52                     reverse=True)
53     total = 0.0
54     for v, w in items:
55         if W <= 0: break
56         take = min(w, W)
57         total += take * (v / w)
58         W -= take
59     return total
60
61 # -- Huffman Encoding -- O(n log n)
62 import heapq
63 def huffman(freq):
64     heap = [[f, [ch, ""]] for ch, f in freq.items()]
65     heapq.heapify(heap)
66     while len(heap) > 1:
67         lo = heapq.heappop(heap)
68         hi = heapq.heappop(heap)
69         for pair in lo[1:]: pair[1] = '0' + pair[1]
70         for pair in hi[1:]: pair[1] = '1' + pair[1]
71         heapq.heappush(heap, [lo[0]+hi[0]] + lo[1:] + hi[1:])
72     return {pair[0]: pair[1] for pair in heap[0][1:]}
73
74 # -- Minimum Spanning Tree cost (Kruskal/Prim) -- see Graph chapter
75
76 # -- Largest Number -- O(n log n)
77 from functools import cmp_to_key
78 def largest_number(nums):
79     nums = list(map(str, nums))
80     nums.sort(key=cmp_to_key(lambda a, b: 1 if a+b > b+a else -1),
81               reverse=True)
82     return "".join(nums).lstrip('0') or "0"
83
84 # -- Candy (minimum candies) -- O(n)
85 def candy(ratings):
86     n = len(ratings)
87     candies = [1] * n
88     for i in range(1, n): # left pass
89         if ratings[i] > ratings[i-1]: candies[i] = candies[i-1] + 1
90     for i in range(n-2, -1, -1): # right pass
91         if ratings[i] > ratings[i+1]: candies[i] = max(candies[i],
92                                                         candies[i+1]+1)
93     return sum(candies)
94
95 # -- Meeting Rooms II (minimum conference rooms) -- O(n log n)
96 def min_meeting_rooms(intervals):
97     intervals.sort(key=lambda x: x[0])
98     heap = [] # min-heap of end times
99     for start, end in intervals:
100         if heap and heap[0] <= start: heapq.heappop(heap)
101         heapq.heappush(heap, end)
102     return len(heap)
103
104 # -- Minimum Cost to Hire K Workers -- O(n log n)

```

```
102 def mincost_to_hire_workers(quality, wage, k):
103     workers = sorted(zip(wage, quality), key=lambda x: x[0]/x[1])
104     max_heap = []      # max-heap of quality (negated)
105     q_sum = 0; ans = float('inf')
106     for w, q in workers:
107         heapq.heappush(max_heap, -q); q_sum += q
108         if len(max_heap) > k: q_sum += heapq.heappop(max_heap)
109         if len(max_heap) == k: ans = min(ans, w/q * q_sum)
110     return ans
```

Divide and Conquer

```

1  # -- Merge Sort (canonical D&C) -- O(n log n)
2  # See Sorting chapter.
3
4  # -- Quick Select (kth smallest) -- O(n) avg
5  def quick_select(nums, k):
6      # Returns kth smallest (1-indexed)
7      def select(lo, hi, k):
8          if lo == hi: return nums[lo]
9          pivot_idx = _partition(nums, lo, hi)
10         rank = pivot_idx - lo + 1
11         if rank == k: return nums[pivot_idx]
12         elif rank > k: return select(lo, pivot_idx-1, k)
13         else: return select(pivot_idx+1, hi, k-rank)
14     return select(0, len(nums)-1, k)
15
16 # -- Maximum Subarray (D&C) -- O(n log n)
17 def max_subarray_dc(arr, lo=None, hi=None):
18     if lo is None: lo, hi = 0, len(arr)-1
19     if lo == hi: return arr[lo]
20     mid = (lo + hi) // 2
21     left = max_subarray_dc(arr, lo, mid)
22     right = max_subarray_dc(arr, mid+1, hi)
23     # Max crossing subarray
24     left_max = curr = 0
25     for i in range(mid, lo-1, -1): curr += arr[i]; left_max = max(
26         left_max, curr)
27     right_max = curr = 0
28     for i in range(mid+1, hi+1): curr += arr[i]; right_max = max(
29         right_max, curr)
30     return max(left, right, left_max + right_max)
31
32 # -- Closest Pair of Points -- O(n log n)
33 def closest_pair(points):
34     import math
35     def dist(p, q): return math.hypot(p[0]-q[0], p[1]-q[1])
36
37     def closest_strip(strip, d):
38         strip.sort(key=lambda p: p[1])
39         min_d = d
40         for i in range(len(strip)):
41             j = i + 1
42             while j < len(strip) and strip[j][1] - strip[i][1] <
43                 min_d:
44                 min_d = min(min_d, dist(strip[i], strip[j])); j += 1
45         return min_d
46
47     def rec(pts):
48         n = len(pts)

```

```

46         if n <= 3:
47             min_d = float('inf')
48             for i in range(n):
49                 for j in range(i+1, n): min_d = min(min_d, dist(pts[
i], pts[j]))
50             return min_d
51         mid = n // 2
52         mid_x = pts[mid][0]
53         d = min(rec(pts[:mid]), rec(pts[mid:]))
54         strip = [p for p in pts if abs(p[0]-mid_x) < d]
55         return min(d, closest_strip(strip, d))
56
57     points.sort()
58     return rec(points)
59
60 # -- Fast Power (Binary Exponentiation) -- O(log n)
61 def fast_pow(base, exp, mod=None):
62     result = 1
63     base = base % mod if mod else base
64     while exp > 0:
65         if exp & 1:
66             result = (result * base) % mod if mod else result * base
67             base = (base * base) % mod if mod else base * base
68             exp >>= 1
69     return result
70
71 # -- Karatsuba Multiplication -- O(n^1.585)
72 def karatsuba(x, y):
73     if x < 10 or y < 10: return x * y
74     n = max(len(str(x)), len(str(y)))
75     m = n // 2
76     high_x, low_x = divmod(x, 10**m)
77     high_y, low_y = divmod(y, 10**m)
78     z0 = karatsuba(low_x, low_y)
79     z2 = karatsuba(high_x, high_y)
80     z1 = karatsuba(low_x + high_x, low_y + high_y) - z0 - z2
81     return z2 * 10**(2*m) + z1 * 10**m + z0

```

Backtracking

```

1  # -- Permutations -- O(n * n!)
2  def permutations(nums):
3      result = []
4      def backtrack(path, used):
5          if len(path) == len(nums): result.append(path[:]); return
6          for i, x in enumerate(nums):
7              if used[i]: continue
8              used[i] = True; path.append(x)
9              backtrack(path, used)
10             path.pop(); used[i] = False
11         backtrack([], [False]*len(nums))
12     return result
13
14 # Permutations with duplicates
15 def permutations_unique(nums):
16     result = []; nums.sort()
17     def backtrack(path, used):
18         if len(path) == len(nums): result.append(path[:]); return
19         for i in range(len(nums)):
20             if used[i]: continue
21             if i > 0 and nums[i] == nums[i-1] and not used[i-1]:
22                 continue
23             used[i] = True; path.append(nums[i])
24             backtrack(path, used)
25             path.pop(); used[i] = False
26         backtrack([], [False]*len(nums))
27     return result
28
29 # -- Subsets -- O(2^n)
30 def subsets(nums):
31     result = []
32     def backtrack(start, path):
33         result.append(path[:])
34         for i in range(start, len(nums)):
35             path.append(nums[i])
36             backtrack(i+1, path)
37             path.pop()
38     backtrack(0, [])
39     return result
40
41 # Subsets with duplicates
42 def subsets_unique(nums):
43     result = []; nums.sort()
44     def backtrack(start, path):
45         result.append(path[:])
46         for i in range(start, len(nums)):
47             if i > start and nums[i] == nums[i-1]: continue
48             path.append(nums[i]); backtrack(i+1, path); path.pop()

```

```

48     backtrack(0, [])
49     return result
50
51 # -- Combinations -- O(C(n,k))
52 def combinations(n, k):
53     result = []
54     def backtrack(start, path):
55         if len(path) == k: result.append(path[:]); return
56         for i in range(start, n+1):
57             path.append(i); backtrack(i+1, path); path.pop()
58     backtrack(1, [])
59     return result
60
61 # Combination Sum (unlimited use, target sum)
62 def combination_sum(candidates, target):
63     result = []; candidates.sort()
64     def backtrack(start, path, remaining):
65         if remaining == 0: result.append(path[:]); return
66         for i in range(start, len(candidates)):
67             if candidates[i] > remaining: break
68             path.append(candidates[i])
69             backtrack(i, path, remaining - candidates[i])    # i not
70             path.pop()
71     backtrack(0, [], target)
72     return result
73
74 # Combination Sum II (each number used once)
75 def combination_sum2(candidates, target):
76     result = []; candidates.sort()
77     def backtrack(start, path, remaining):
78         if remaining == 0: result.append(path[:]); return
79         for i in range(start, len(candidates)):
80             if candidates[i] > remaining: break
81             if i > start and candidates[i] == candidates[i-1]:
82                 continue
83             path.append(candidates[i])
84             backtrack(i+1, path, remaining - candidates[i])
85             path.pop()
86     backtrack(0, [], target)
87     return result
88
89 # -- N-Queens -- O(n!)
90 def solve_n_queens(n):
91     result = []
92     cols = set(); diag1 = set(); diag2 = set()    # diagonals
93
94     def backtrack(row, board):
95         if row == n: result.append("".join(r) for r in board); return
96         for col in range(n):
97             if col in cols or (row-col) in diag1 or (row+col) in
98                 diag2: continue
99             cols.add(col); diag1.add(row-col); diag2.add(row+col)
100             board[row][col] = 'Q'
101             backtrack(row+1, board)
102             board[row][col] = '.'; cols.discard(col)

```

```

101         diag1.discard(row-col); diag2.discard(row+col)
102
103     board = [['.'] * n for _ in range(n)]
104     backtrack(0, board)
105     return result
106
107 # -- Sudoku Solver -- O(9^(n*n)) but in practice much faster
108 def solve_sudoku(board):
109     empty = [(r, c) for r in range(9) for c in range(9) if board[r][
110 c] == '.']
111
112     def is_valid(r, c, ch):
113         if ch in board[r]: return False
114         if ch in [board[i][c] for i in range(9)]: return False
115         br, bc = 3*(r//3), 3*(c//3)
116         for i in range(br, br+3):
117             for j in range(bc, bc+3):
118                 if board[i][j] == ch: return False
119         return True
120
121     def backtrack(idx):
122         if idx == len(empty): return True
123         r, c = empty[idx]
124         for ch in '123456789':
125             if is_valid(r, c, ch):
126                 board[r][c] = ch
127                 if backtrack(idx+1): return True
128                 board[r][c] = '.'
129         return False
130
131     backtrack(0)
132
133 # -- Word Search in Grid -- O(m*n*4^L)
134 def word_search(board, word):
135     m, n = len(board), len(board[0])
136     def dfs(r, c, idx):
137         if idx == len(word): return True
138         if r < 0 or r >= m or c < 0 or c >= n: return False
139         if board[r][c] != word[idx]: return False
140         tmp = board[r][c]; board[r][c] = '#'
141         found = any(dfs(r+dr, c+dc, idx+1) for dr, dc in [(-1,0),
142 (1,0), (0,-1), (0,1)])
143         board[r][c] = tmp
144         return found
145     return any(dfs(r, c, 0) for r in range(m) for c in range(n))
146
147 # -- Letter Combinations of Phone Number -- O(4^n)
148 def letter_combinations(digits):
149     if not digits: return []
150     mapping = {'2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl',
151 '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz'}
152     result = []
153     def backtrack(idx, path):
154         if idx == len(digits): result.append("".join(path)); return
155         for ch in mapping[digits[idx]]:
156             path.append(ch); backtrack(idx+1, path); path.pop()
157     backtrack(0, [])

```

```
156     return result
```

Part VIII

String and Mathematical Algorithms

String Algorithms

26.1 KMP Pattern Matching

```

1  # -- KMP (Knuth-Morris-Pratt) --  $O(n + m)$ 
2  def kmp_search(text, pattern):
3      def build_lps(pattern):
4          m = len(pattern)
5          lps = [0] * m          # longest proper prefix which is also
                                   suffix
6          length = 0; i = 1
7          while i < m:
8              if pattern[i] == pattern[length]:
9                  length += 1; lps[i] = length; i += 1
10             else:
11                 if length != 0: length = lps[length-1]
12                 else:          lps[i] = 0; i += 1
13         return lps
14
15     n, m = len(text), len(pattern)
16     if m == 0: return []
17     lps = build_lps(pattern)
18     result = []; i = j = 0
19     while i < n:
20         if text[i] == pattern[j]:
21             i += 1; j += 1
22         if j == m:
23             result.append(i - j)
24             j = lps[j-1]
25         elif i < n and text[i] != pattern[j]:
26             if j != 0: j = lps[j-1]
27             else:     i += 1
28     return result          # list of start indices
29
30 # -- Z-Function --  $O(n)$ 
31 def z_function(s):
32     n = len(s)
33     z = [0] * n
34     z[0] = n
35     l = r = 0
36     for i in range(1, n):
37         if i < r: z[i] = min(r-i, z[i-l])
38         while i + z[i] < n and s[z[i]] == s[i + z[i]]:
39             z[i] += 1
40         if i + z[i] > r: l, r = i, i + z[i]
41     return z
42
43 def z_search(text, pattern):
44     s = pattern + '#' + text

```

```

45     z = z_function(s)
46     m = len(pattern)
47     return [i - m - 1 for i in range(m+1, len(s)) if z[i] == m]

```

26.2 Rabin-Karp and Rolling Hash

```

1  # -- Rabin-Karp -- O(n + m) avg, O(nm) worst
2  def rabin_karp(text, pattern):
3      n, m = len(text), len(pattern)
4      if m > n: return []
5      MOD = (1 << 61) - 1      # Mersenne prime
6      BASE = 131
7      pw = pow(BASE, m, MOD)
8
9      def h(s): return sum(ord(c) * pow(BASE, i, MOD) for i, c in
10         enumerate(s)) % MOD
11
12     p_hash = h(pattern)
13     t_hash = h(text[:m])
14     result = []
15     if t_hash == p_hash and text[:m] == pattern: result.append(0)
16
17     for i in range(1, n - m + 1):
18         t_hash = (t_hash - ord(text[i-1]) + MOD) % MOD
19         t_hash = t_hash * BASE % MOD
20         t_hash = (t_hash + ord(text[i+m-1]) * pow(BASE, m-1, MOD)) %
21         MOD
22         if t_hash == p_hash and text[i:i+m] == pattern:
23             result.append(i)
24     return result
25
26 # -- Polynomial Rolling Hash for substring comparison -- O(n) build,
27 # O(1) query
28 class RollingHash:
29     def __init__(self, s, base=131, mod=(1<<61)-1):
30         n = len(s)
31         self.mod = mod
32         self.pw = [1] * (n+1)
33         self.h = [0] * (n+1)
34         for i in range(n):
35             self.h[i+1] = (self.h[i] * base + ord(s[i])) % mod
36             self.pw[i+1] = self.pw[i] * base % mod
37
38     def query(self, l, r):      # hash of s[l..r] inclusive, O(1)
39         return (self.h[r+1] - self.h[l] * self.pw[r-l+1]) % self.mod

```

26.3 Suffix Array and LCP Array

```

1  # -- Suffix Array (O(n log^2 n) simple version)
2  def build_suffix_array(s):
3      s += '$'      # sentinel (lexicographically smallest)
4      n = len(s)
5      sa = sorted(range(n), key=lambda i: s[i:])

```

```

6     return sa[1:]      # exclude sentinel
7
8     # -- LCP Array (Kasai's algorithm) -- O(n)
9     def build_lcp(s, sa):
10         n = len(s)
11         rank = [0] * n
12         for i, suf in enumerate(sa): rank[suf] = i
13         lcp = [0] * n
14         h = 0
15         for i in range(n):
16             if rank[i] > 0:
17                 j = sa[rank[i]-1]
18                 while i+h < n and j+h < n and s[i+h] == s[j+h]: h += 1
19                 lcp[rank[i]] = h
20                 if h > 0: h -= 1
21         return lcp
22
23     # -- Longest Repeated Substring using SA + LCP -- O(n)
24     def longest_repeated_substring(s):
25         sa = build_suffix_array(s)
26         lcp = build_lcp(s, sa)
27         max_lcp = max(lcp)
28         idx = lcp.index(max_lcp)
29         return s[sa[idx]: sa[idx] + max_lcp]

```

26.4 Manacher's Algorithm

```

1     # -- Manacher -- longest palindromic substring -- O(n)
2     def manacher(s):
3         T = '#' + '#'.join(s) + '#'
4         n = len(T)
5         P = [0] * n
6         C = R = 0
7         for i in range(n):
8             mirror = 2*C - i
9             if i < R: P[i] = min(R-i, P[mirror])
10            while i+P[i]+1 < n and i-P[i]-1 >= 0 and T[i+P[i]+1] == T[i-
P[i]-1]:
11                P[i] += 1
12                if i+P[i] > R: C, R = i, i+P[i]
13            max_len = max(P)
14            center = P.index(max_len)
15            start = (center - max_len) // 2
16            return s[start: start + max_len]
17
18     # Count all palindromic substrings using Manacher
19     def count_palindromes(s):
20         T = '#' + '#'.join(s) + '#'
21         n = len(T); P = [0]*n; C = R = 0
22         for i in range(n):
23             mirror = 2*C - i
24             if i < R: P[i] = min(R-i, P[mirror])
25             while i+P[i]+1 < n and i-P[i]-1 >= 0 and T[i+P[i]+1] == T[i-
P[i]-1]:
26                 P[i] += 1

```

```

27         if i+P[i] > R: C, R = i, i+P[i]
28     return sum((p+1)//2 for p in P)

```

26.5 Aho-Corasick Multi-Pattern Search

```

1  # -- Aho-Corasick -- O(n + m + z), z = total matches
2  from collections import deque, defaultdict
3
4  class AhoCorasick:
5      def __init__(self, words):
6          self.goto = [{}]
7          self.fail = [0]
8          self.output = [[]]
9          self._build(words)
10
11     def _build(self, words):
12         for word in words:
13             cur = 0
14             for ch in word:
15                 if ch not in self.goto[cur]:
16                     self.goto.append({}); self.fail.append(0); self.
output.append([])
17                     self.goto[cur][ch] = len(self.goto) - 1
18                     cur = self.goto[cur][ch]
19                     self.output[cur].append(word)
20             # BFS to set failure links
21             queue = deque()
22             for ch, s in self.goto[0].items():
23                 queue.append(s)
24             while queue:
25                 r = queue.popleft()
26                 for ch, s in self.goto[r].items():
27                     queue.append(s)
28                     state = self.fail[r]
29                     while state and ch not in self.goto[state]: state =
self.fail[state]
30                     self.fail[s] = self.goto[state].get(ch, 0)
31                     if self.fail[s] == s: self.fail[s] = 0
32                     self.output[s] += self.output[self.fail[s]]
33
34     def search(self, text):
35         cur = 0; results = []
36         for i, ch in enumerate(text):
37             while cur and ch not in self.goto[cur]: cur = self.fail[
cur]
38             cur = self.goto[cur].get(ch, 0)
39             for word in self.output[cur]:
40                 results.append((i - len(word) + 1, word))
41         return results

```

Mathematical Algorithms

27.1 Number Theory

```

1 import math
2
3 # -- GCD / LCM --  $O(\log(\min(a,b)))$ 
4 def gcd(a, b): return math.gcd(a, b)           # or: while b: a,b=b,
        a%b
5 def lcm(a, b): return a * b // math.gcd(a, b)
6
7 # -- Extended Euclidean Algorithm --  $O(\log(\min(a,b)))$ 
8 def extended_gcd(a, b):
9     # Returns (gcd, x, y) such that  $a*x + b*y = \text{gcd}$ 
10    if a == 0: return b, 0, 1
11    g, x1, y1 = extended_gcd(b % a, a)
12    return g, y1 - (b//a)*x1, x1
13
14 # -- Modular Inverse --  $O(\log m)$ 
15 def mod_inverse(a, m):
16     g, x, _ = extended_gcd(a % m, m)
17     if g != 1: raise ValueError("Inverse doesn't exist")
18     return (x % m + m) % m
19
20 # Python 3.8+ shorthand:
21 # pow(a, -1, m)
22
23 # -- Sieve of Eratosthenes --  $O(n \log \log n)$ 
24 def sieve(n):
25     is_prime = [True] * (n+1)
26     is_prime[0] = is_prime[1] = False
27     for i in range(2, int(n**0.5)+1):
28         if is_prime[i]:
29             for j in range(i*i, n+1, i): is_prime[j] = False
30     return [i for i in range(2, n+1) if is_prime[i]]
31
32 # -- Segmented Sieve --  $O(n)$  for primes up to n
33 def segmented_sieve(lo, hi):
34     small_primes = sieve(int(hi**0.5)+1)
35     is_prime = [True] * (hi - lo + 1)
36     if lo == 1: is_prime[0] = False
37     for p in small_primes:
38         start = max(p*p, ((lo + p - 1) // p) * p)
39         for j in range(start, hi+1, p): is_prime[j-lo] = False
40     return [i+lo for i, v in enumerate(is_prime) if v]
41
42 # -- Linear Sieve (Euler's sieve) --  $O(n)$  strict
43 def linear_sieve(n):
44     primes = []

```

```

45     is_comp = [False] * (n+1)
46     for i in range(2, n+1):
47         if not is_comp[i]: primes.append(i)
48         for p in primes:
49             if i * p > n: break
50             is_comp[i*p] = True
51             if i % p == 0: break
52     return primes
53
54 # -- Prime Factorisation -- O(sqrt(n))
55 def prime_factors(n):
56     factors = {}
57     d = 2
58     while d * d <= n:
59         while n % d == 0: factors[d] = factors.get(d,0)+1; n //= d
60         d += 1
61     if n > 1: factors[n] = factors.get(n,0)+1
62     return factors
63
64 # -- Euler's Totient Function -- O(sqrt(n)) for single n
65 def totient(n):
66     result = n
67     p = 2
68     while p*p <= n:
69         if n % p == 0:
70             while n % p == 0: n //= p
71             result -= result // p
72         p += 1
73     if n > 1: result -= result // n
74     return result
75
76 # Totient sieve -- O(n log log n) for all values up to n
77 def totient_sieve(n):
78     phi = list(range(n+1))
79     for i in range(2, n+1):
80         if phi[i] == i: # i is prime
81             for j in range(i, n+1, i): phi[j] -= phi[j] // i
82     return phi
83
84 # -- Fast Power (Binary Exponentiation) -- O(log n)
85 def fast_pow(b, e, mod=None):
86     r = 1
87     b = b % mod if mod else b
88     while e:
89         if e & 1: r = r*b%mod if mod else r*b
90         b = b*b%mod if mod else b*b
91         e >>= 1
92     return r
93
94 # -- Matrix Exponentiation -- O(k^3 log n)
95 def mat_mul(A, B, mod):
96     k = len(A)
97     C = [[0]*k for _ in range(k)]
98     for i in range(k):
99         for j in range(k):
100             for l in range(k):
101                 C[i][j] = (C[i][j] + A[i][l]*B[l][j]) % mod

```

```

102     return C
103
104 def mat_pow(M, n, mod):
105     k = len(M)
106     R = [[int(i==j) for j in range(k)] for i in range(k)] #
107     identity
108     while n:
109         if n & 1: R = mat_mul(R, M, mod)
110         M = mat_mul(M, M, mod); n >>= 1
111     return R
112
113 # Fibonacci with matrix exponentiation -- O(log n)
114 def fib_matrix(n, mod=10**9+7):
115     if n <= 1: return n
116     M = [[1,1],[1,0]]
117     R = mat_pow(M, n-1, mod)
118     return R[0][0]
119
120 # -- Chinese Remainder Theorem (CRT) -- O(log(min(m1,m2)))
121 def crt(r1, m1, r2, m2):
122     # Find x such that x = r1 (mod m1), x = r2 (mod m2)
123     g, p, q = extended_gcd(m1, m2)
124     if (r2 - r1) % g != 0: return None, None # no solution
125     lcm_ = m1 * m2 // g
126     x = (r1 + m1 * ((r2-r1)//g * p % (m2//g))) % lcm_
127     return x, lcm_
128
129 # -- Lucas's Theorem -- n choose r mod prime p
130 def lucas_ncr(n, r, p):
131     if r == 0: return 1
132     ni, ri = n % p, r % p
133     if ri > ni: return 0
134     return ncr(ni, ri, p) * lucas_ncr(n//p, r//p, p) % p
135
136 # Precompute factorials for nCr mod p
137 def precompute_factorials(n, mod):
138     fact = [1]*(n+1)
139     inv_f = [1]*(n+1)
140     for i in range(1, n+1): fact[i] = fact[i-1]*i % mod
141     inv_f[n] = pow(fact[n], mod-2, mod)
142     for i in range(n-1, -1, -1): inv_f[i] = inv_f[i+1]*(i+1) % mod
143     def ncr(n, r):
144         if r < 0 or r > n: return 0
145         return fact[n] * inv_f[r] % mod * inv_f[n-r] % mod
146     return ncr

```

27.2 Bit Manipulation

```

1 # -- Bit operations in Python
2
3 x = 42                                # 0b101010
4 x & y                                # AND
5 x | y                                # OR
6 x ^ y                                # XOR

```

```

6 ~x                                # NOT (= -(x+1) in Python!)
7 x << k                            # left shift by k
8 x >> k                            # right shift by k (arithmetic)
9
10 # Python integers are arbitrary precision -- no overflow!
11
12 # -- Common tricks
13
13 x & 1                             # check if odd
14 x & (x-1)                         # clear lowest set bit; 0 iff power
    of 2
15 x & (-x)                         # isolate lowest set bit
16 x | (x-1)                        # set all bits below lowest set bit
17 x.bit_length()                   # number of bits (= floor(log2(x)) +
    1)
18 bin(x).count('1')                # popcount (number of 1-bits)
19 int.bit_count(x)                 # Python 3.10+ native popcount
20
21 # Check if power of 2
22 def is_power_of_two(n): return n > 0 and (n & (n-1)) == 0
23
24 # Next power of 2
25 def next_pow2(n):
26     p = 1
27     while p < n: p <= 1
28     return p
29
30 # -- XOR tricks
31
31 # Single number (one element appears odd times)
32 def single_number(nums):
33     result = 0
34     for x in nums: result ^= x
35     return result
36
37 # Single number II (all others appear 3x) -- O(n)
38 def single_number_3x(nums):
39     ones = twos = 0
40     for x in nums:
41         ones = (ones ^ x) & ~twos
42         twos = (twos ^ x) & ~ones
43     return ones
44
45 # Two single numbers (two elements appear once)
46 def single_number_two(nums):
47     xor = 0
48     for x in nums: xor ^= x
49     diff_bit = xor & (-xor)       # rightmost differing bit
50     a = b = 0
51     for x in nums:
52         if x & diff_bit: a ^= x
53         else:             b ^= x
54     return a, b
55
56 # -- Bit DP: bitmask DP -- O(2^n * n)

```

```

57 # Travelling Salesman (held-karp)
58 def tsp(dist, n):
59     INF = float('inf')
60     dp = [[INF]*(1<n) for _ in range(n)]
61     dp[0][1] = 0 # start at city 0, visited = {0}
62     for mask in range(1, 1<n):
63         for u in range(n):
64             if not (mask >> u & 1): continue
65             if dp[u][mask] == INF: continue
66             for v in range(n):
67                 if mask >> v & 1: continue
68                 new_mask = mask | (1<v)
69                 dp[v][new_mask] = min(dp[v][new_mask], dp[u][mask] +
dist[u][v])
70     full = (1<n) - 1
71     return min(dp[u][full] + dist[u][0] for u in range(1, n))
72
73 # Count subsets with given XOR
74 def count_subsets_xor(arr, x):
75     from functools import cache
76     n = len(arr)
77     @cache
78     def dp(i, cur_xor):
79         if i == n: return 1 if cur_xor == x else 0
80         return dp(i+1, cur_xor^arr[i]) + dp(i+1, cur_xor)
81     return dp(0, 0)
82
83 # -- Subset Enumeration using bitmask -- O(2^n)
84 def all_subsets_bitmask(arr):
85     n = len(arr)
86     for mask in range(1 << n):
87         subset = [arr[i] for i in range(n) if mask >> i & 1]
88         print(subset)
89
90 # Enumerate all subsets of a given mask (for SOS DP)
91 def enumerate_submasks(mask):
92     sub = mask
93     while sub:
94         yield sub
95         sub = (sub-1) & mask
96
97 # -- Sum over Subsets (SOS) DP -- O(2^n * n)
98 def sos_dp(arr):
99     n = max(arr).bit_length()
100     dp = [0] * (1 << n)
101     for x in arr: dp[x] += 1
102     for i in range(n):
103         for mask in range(1 << n):
104             if mask >> i & 1:
105                 dp[mask] += dp[mask ^ (1 << i)]
106     return dp # dp[mask] = sum of arr[x] for all x subset of mask

```

27.3 Geometry

```

1 # -- 2D Point / Vector operations -- O(1)

```

```

2 def cross(o, a, b):
3     # Cross product of vectors OA and OB
4     # > 0: counterclockwise, < 0: clockwise, = 0: collinear
5     return (a[0]-o[0])*(b[1]-o[1]) - (a[1]-o[1])*(b[0]-o[0])
6
7 def dot(a, b): return a[0]*b[0] + a[1]*b[1]
8 def dist2(a, b): return (a[0]-b[0])**2 + (a[1]-b[1])**2
9
10 # -- Convex Hull (Graham Scan) -- O(n log n)
11 def convex_hull(points):
12     points = sorted(set(points))
13     if len(points) <= 1: return points
14     # Build lower hull
15     lower = []
16     for p in points:
17         while len(lower) >= 2 and cross(lower[-2], lower[-1], p) <=
18 0:
19             lower.pop()
20         lower.append(p)
21     # Build upper hull
22     upper = []
23     for p in reversed(points):
24         while len(upper) >= 2 and cross(upper[-2], upper[-1], p) <=
25 0:
26             upper.pop()
27         upper.append(p)
28     return lower[:-1] + upper[::-1]
29
30 # -- Point in Polygon (Ray Casting) -- O(n)
31 def point_in_polygon(point, polygon):
32     x, y = point; inside = False
33     n = len(polygon); j = n - 1
34     for i in range(n):
35         xi, yi = polygon[i]; xj, yj = polygon[j]
36         if ((yi > y) != (yj > y)) and (x < (xj-xi)*(y-yi)/(yj-yi)+xi
37 ):
38             inside = not inside
39         j = i
40     return inside
41
42 # -- Line Segment Intersection -- O(1)
43 def segments_intersect(p1, p2, p3, p4):
44     d1 = cross(p3, p4, p1); d2 = cross(p3, p4, p2)
45     d3 = cross(p1, p2, p3); d4 = cross(p1, p2, p4)
46     if ((d1 > 0 and d2 < 0) or (d1 < 0 and d2 > 0)) and \
47         ((d3 > 0 and d4 < 0) or (d3 < 0 and d4 > 0)):
48         return True
49     # Check collinear cases (omitted for brevity)
50     return False

```

Part IX

Advanced Topics and Competitive Programming

Competitive Programming Patterns

28.1 Two Heaps Pattern

```

1  # -- Find Median from Data Stream --  $O(\log n)$  per add,  $O(1)$  query
2  import heapq
3  class MedianFinder:
4      def __init__(self):
5          self.lo = []      # max-heap (negated) for lower half
6          self.hi = []      # min-heap for upper half
7
8      def add_num(self, num):
9          heapq.heappush(self.lo, -num)
10         # Ensure lo's max <= hi's min
11         if self.hi and -self.lo[0] > self.hi[0]:
12             heapq.heappush(self.hi, -heapq.heappop(self.lo))
13         # Balance sizes: lo can have at most 1 extra
14         if len(self.lo) > len(self.hi) + 1:
15             heapq.heappush(self.hi, -heapq.heappop(self.lo))
16         elif len(self.hi) > len(self.lo):
17             heapq.heappush(self.lo, -heapq.heappop(self.hi))
18
19     def find_median(self):
20         if len(self.lo) > len(self.hi): return float(-self.lo[0])
21         return (-self.lo[0] + self.hi[0]) / 2.0

```

28.2 Intervals

```

1  # -- Merge Overlapping Intervals --  $O(n \log n)$ 
2  def merge_intervals(intervals):
3      intervals.sort(); result = [intervals[0]]
4      for start, end in intervals[1:]:
5          if start <= result[-1][1]: result[-1][1] = max(result
6          [-1][1], end)
7          else:
8              result.append([start, end])
9      return result
10
11 # -- Insert Interval --  $O(n)$ 
12 def insert_interval(intervals, new):
13     result = []; i = 0; n = len(intervals)
14     # Add all intervals ending before new starts
15     while i < n and intervals[i][1] < new[0]: result.append(
16     intervals[i]); i += 1
17     # Merge overlapping
18     while i < n and intervals[i][0] <= new[1]:
19         new[0] = min(new[0], intervals[i][0])
20         new[1] = max(new[1], intervals[i][1]); i += 1

```

```

18     result.append(new)
19     result.extend(intervals[i:])
20     return result
21
22 # -- Non-overlapping Intervals (min removals) -- O(n log n)
23 def erase_overlap_intervals(intervals):
24     intervals.sort(key=lambda x: x[1])
25     last_end = float('-inf'); removed = 0
26     for start, end in intervals:
27         if start >= last_end: last_end = end
28         else: removed += 1
29     return removed

```

28.3 Monotonic Queue / Stack Patterns

```

1 # -- Largest Rectangle in Histogram (revisited) -- O(n)
2 def largest_rectangle_histogram(heights):
3     heights = heights + [0]; stack = [-1]; max_area = 0
4     for i, h in enumerate(heights):
5         while stack[-1] != -1 and heights[stack[-1]] >= h:
6             height = heights[stack.pop()]
7             width = i - stack[-1] - 1
8             max_area = max(max_area, height * width)
9         stack.append(i)
10    return max_area
11
12 # -- Jump Game VI (max score with window k) -- O(n log k) with heap
13 def max_result(nums, k):
14     from collections import deque
15     n = len(nums)
16     dq = deque([0]) # indices, decreasing dp value
17     dp = [0] * n; dp[0] = nums[0]
18     for i in range(1, n):
19         while dq and dq[0] < i - k: dq.popleft()
20         dp[i] = dp[dq[0]] + nums[i]
21         while dq and dp[dq[-1]] <= dp[i]: dq.pop()
22         dq.append(i)
23     return dp[n-1]

```

28.4 Graph Colouring and Matching

```

1 # -- Hungarian Algorithm (assignment problem) -- O(n^3)
2 def hungarian(cost):
3     n = len(cost)
4     u = [0]*(n+1); v = [0]*(n+1)
5     p = [0]*(n+1); way = [0]*(n+1)
6     for i in range(1, n+1):
7         p[0] = i; j0 = 0
8         minv = [float('inf')]*(n+1)
9         used = [False]*(n+1)
10        while True:
11            used[j0] = True; i0 = p[j0]; delta = float('inf'); j1 =
-1

```

```
12         for j in range(1, n+1):
13             if not used[j]:
14                 cur = cost[i0-1][j-1] - u[i0] - v[j]
15                 if cur < minv[j]: minv[j] = cur; way[j] = j0
16                 if minv[j] < delta: delta = minv[j]; j1 = j
17             for j in range(n+1):
18                 if used[j]: u[p[j]] += delta; v[j] -= delta
19                 else: minv[j] -= delta
20             j0 = j1
21             if p[j0] == 0: break
22         while j0:
23             p[j0] = p[way[j0]]; j0 = way[j0]
24     ans = [-u[0]] # total cost
25     assignment = [0]*n
26     for j in range(1, n+1):
27         if p[j]: assignment[p[j]-1] = j-1
28     return -u[0], assignment
```

Miscellaneous Algorithms and Tricks

29.1 Reservoir Sampling

```
1 # -- Reservoir Sampling: pick k items uniformly at random -- O(n)
2 import random
3 def reservoir_sample(stream, k):
4     reservoir = []
5     for i, item in enumerate(stream):
6         if i < k:
7             reservoir.append(item)
8         else:
9             j = random.randint(0, i)
10            if j < k: reservoir[j] = item
11    return reservoir
```

29.2 Fisher-Yates Shuffle

```
1 # -- Fisher-Yates Shuffle -- O(n), uniform distribution
2 def shuffle(arr):
3     n = len(arr)
4     for i in range(n-1, 0, -1):
5         j = random.randint(0, i)
6         arr[i], arr[j] = arr[j], arr[i]
7     return arr
```

29.3 Meeting in the Middle

```
1 # -- 4-sum count (meet-in-the-middle) -- O(n^2)
2 def four_sum_count(A, B, C, D):
3     from collections import Counter
4     ab_sums = Counter(a+b for a in A for b in B)
5     return sum(ab_sums.get(-(c+d), 0) for c in C for d in D)
```

29.4 Mo's Algorithm

```
1 # -- Mo's Algorithm: offline range queries -- O((n+q)*sqrt(n))
2 import math
3 def mo_algorithm(arr, queries):
4     n = len(arr); block = max(1, int(math.sqrt(n)))
5     # Sort queries: (l, r, idx) by block of l, then r
6     queries = sorted(enumerate(queries), key=lambda e: (e[1][0]//
    block, e[1][1]))
```

```

7
8     cur_l = cur_r = 0
9     cur_sum = arr[0]
10    results = [0] * len(queries)
11
12    def add(pos): nonlocal cur_sum; cur_sum += arr[pos]
13    def remove(pos): nonlocal cur_sum; cur_sum -= arr[pos]
14
15    for idx, (l, r) in queries:
16        while cur_r < r: cur_r += 1; add(cur_r)
17        while cur_l > l: cur_l -= 1; add(cur_l)
18        while cur_r > r: remove(cur_r); cur_r -= 1
19        while cur_l < l: remove(cur_l); cur_l += 1
20        results[idx] = cur_sum
21
22    return results

```

29.5 Heavy-Light Decomposition (HLD)

```

1  # -- Heavy-Light Decomposition -- O(n) build, O(log^2 n) path query
2  class HLD:
3      def __init__(self, adj, values, root=0):
4          n = self.n = len(adj)
5          self.parent = [-1]*n; self.depth = [0]*n
6          self.heavy = [-1]*n; self.head = [0]*n
7          self.pos = [0]*n; self.sz = [1]*n
8          self._dfs_sz(root, -1, adj)
9          self._dfs_hld(root, root, adj, [0])
10         # Build segment tree on HLD positions
11         arr = [values[i] for i in range(n)] # reorder by pos
12         self.seg = SegTree(arr)
13
14     def _dfs_sz(self, u, p, adj):
15         self.parent[u] = p; heavy_sz = 0
16         for v in adj[u]:
17             if v == p: continue
18             self.depth[v] = self.depth[u]+1
19             self._dfs_sz(v, u, adj)
20             self.sz[u] += self.sz[v]
21             if self.sz[v] > heavy_sz: heavy_sz = self.sz[v]; self.
heavy[u] = v
22
23     def _dfs_hld(self, u, h, adj, cur):
24         self.head[u] = h; self.pos[u] = cur[0]; cur[0] += 1
25         if self.heavy[u] != -1: self._dfs_hld(self.heavy[u], h, adj,
cur)
26         for v in adj[u]:
27             if v != self.parent[u] and v != self.heavy[u]:
28                 self._dfs_hld(v, v, adj, cur)
29
30     def path_query(self, u, v):
31         result = 0
32         while self.head[u] != self.head[v]:
33             if self.depth[self.head[u]] < self.depth[self.head[v]]:
u, v = v, u

```

```

34         result += self.seg.query(self.pos[self.head[u]], self.
pos[u])
35         u = self.parent[self.head[u]]
36         if self.depth[u] > self.depth[v]: u, v = v, u
37         result += self.seg.query(self.pos[u], self.pos[v])
38         return result

```

29.6 Centroid Decomposition

```

1  # -- Centroid Decomposition -- O(n log n) build, O(log n) per query
2  class CentroidDecomp:
3      def __init__(self, adj, n):
4          self.adj = adj; self.n = n
5          self.sz = [0]*n; self.removed = [False]*n
6          self.parent = [-1]*n
7
8      def _get_sz(self, u, p):
9          self.sz[u] = 1
10         for v in self.adj[u]:
11             if v != p and not self.removed[v]:
12                 self._get_sz(v, u); self.sz[u] += self.sz[v]
13
14     def _get_centroid(self, u, p, tree_sz):
15         for v in self.adj[u]:
16             if v != p and not self.removed[v]:
17                 if self.sz[v] > tree_sz // 2:
18                     return self._get_centroid(v, u, tree_sz)
19         return u
20
21     def build(self, u=0, p=-1):
22         self._get_sz(u, -1)
23         c = self._get_centroid(u, -1, self.sz[u])
24         self.removed[c] = True
25         self.parent[c] = p
26         for v in self.adj[c]:
27             if not self.removed[v]: self.build(v, c)
28         self.removed[c] = False
29         return c

```

Input / Output Optimisation

```

1  import sys
2  input = sys.stdin.readline      # 3-5x faster than built-in input()
3
4  # -- Fast read of integers
5  def read_int():      return int(sys.stdin.readline())
6  def read_ints():     return list(map(int, sys.stdin.readline().split
7  ()))
8  def read_matrix(m): return [list(map(int, sys.stdin.readline().split
9  ())) for _ in range(m)]
10
11 # -- Fast output (buffer before printing)
12 out = []
13 out.append(str(answer))
14 print('\n'.join(out))          # single system call vs n print()
15                                 calls
16
17 # -- Or use sys.stdout.write
18 sys.stdout.write('\n'.join(map(str, results)) + '\n')
19
20 # -- PyPy tips (if available):
21 # 1. Use sys.stdin.read() and split once upfront
22 # 2. Avoid excessive function calls
23 # 3. Use local variables in tight loops (faster than global)
24
25 # -- Complete fast I/O template
26 import sys
27 from collections import defaultdict, deque
28 from heapq import *
29 from bisect import *
30 from itertools import *
31 from functools import cache
32 input = sys.stdin.readline
33 MOD = 10**9 + 7
34 INF = float('inf')
35
36 def solve():
37     n = int(input())
38     arr = list(map(int, input().split()))
39     # ... solution ...
40
41 T = int(input())
42 for _ in range(T):
43     solve()

```

Complete Algorithm Reference Table

Algorithm / DS	Time	Space	Notes
Linear Structures			
Array access	$O(1)$	—	Random access
Array search	$O(n)$	—	Linear scan
Dynamic array push	$O(1)$ amort	$O(n)$	Python list
Linked list insert	$O(1)$	$O(n)$	With pointer
Stack push/pop	$O(1)$	$O(n)$	
Queue enqueue/deq	$O(1)$	$O(n)$	deque
Trees			
BST search/insert	$O(\log n)$ avg	$O(n)$	$O(n)$ worst
AVL search/insert	$O(\log n)$	$O(n)$	Guaranteed
Heap push/pop	$O(\log n)$	$O(n)$	
Heap build	$O(n)$	$O(n)$	
Trie insert/search	$O(m)$	$O(nm)$	m = key length
Segment tree query	$O(\log n)$	$O(4n)$	
BIT update/query	$O(\log n)$	$O(n)$	
Sparse table query	$O(1)$	$O(n \log n)$	Static only
Hashing			
Dict get/set/del	$O(1)$ avg	$O(n)$	$O(n)$ worst
Set add/in/remove	$O(1)$ avg	$O(n)$	
Sorting			
Comparison sort	$O(n \log n)$	$O(\log n)$	Lower bound
Counting sort	$O(n + k)$	$O(k)$	Integer keys
Radix sort	$O(dn)$	$O(n + b)$	d digits, base b
Graph			
DFS / BFS	$O(V + E)$	$O(V)$	
Dijkstra	$O((V + E) \log V)$	$O(V)$	Non-neg weights
Bellman-Ford	$O(VE)$	$O(V)$	Neg weights
Floyd-Warshall	$O(V^3)$	$O(V^2)$	All-pairs
Kruskal MST	$O(E \log E)$	$O(V)$	+ DSU
Prim MST	$O((V + E) \log V)$	$O(V)$	
Topo Sort	$O(V + E)$	$O(V)$	DAG only
Tarjan SCC	$O(V + E)$	$O(V)$	
Bridges/APs	$O(V + E)$	$O(V)$	
Strings			
KMP	$O(n + m)$	$O(m)$	
Z-function	$O(n)$	$O(n)$	

Algorithm / DS	Time	Space	Notes
Rabin-Karp	$O(n + m)$ avg	$O(1)$	Palindromes
Manacher	$O(n)$	$O(n)$	
Suffix Array	$O(n \log^2 n)$	$O(n)$	
Aho-Corasick	$O(n + m + z)$	$O(nm)$	$z = \text{matches}$
DP			
1D DP	$O(n) - O(n^2)$	$O(n)$	Optimised
2D DP (LCS, etc.)	$O(mn)$	$O(\min(m, n))$	
LIS	$O(n \log n)$	$O(n)$	
Knapsack 0/1	$O(nW)$	$O(W)$	Patience sort
Interval DP	$O(n^3)$	$O(n^2)$	
Bitmask DP	$O(2^n \cdot n)$	$O(2^n)$	
Digit DP	$O(D \cdot S)$	$O(D \cdot S)$	
Number Theory			
GCD	$O(\log n)$	$O(1)$	$k \times k$ matrix
Sieve of Eratosthenes	$O(n \log \log n)$	$O(n)$	
Fast power	$O(\log n)$	$O(1)$	
Matrix power	$O(k^3 \log n)$	$O(k^2)$	
Special			
DSU find/union	$O(\alpha(n))$	$O(n)$	Nearly $O(1)$
Mo's Algorithm	$O((n + q)\sqrt{n})$	$O(n)$	Offline queries

Common Pitfalls: C++ to Python

Top 10 Pitfalls when Moving from C++ to Python DSA

1. **Recursion limit:** Python default is 1000. Add `sys.setrecursionlimit(300000)` or convert to iterative.
2. **Max-heap:** Python only has min-heap. Negate values: `heapq.heappush(h, -x)`.
3. **Integer type:** Python integers never overflow (arbitrary precision). No need for `long long`.
4. **Mutable default args:** `def f(arr=[]):` shares the list across calls. Use `def f(arr=None): if arr is None: arr = []`
5. **2D list initialisation:** `[[0]*n]*m` creates m references to the same row. Always use `[[0]*n for _ in range(m)]`.
6. **String concatenation:** `s += char` in a loop is $O(n^2)$. Use `"".join(list)`.
7. **Slow `list.pop(0)`:** $O(n)$. Use `collections.deque` for $O(1)$ front pops.
8. **No built-in balanced BST:** Use `sortedcontainers.SortedList` (pip install) or simulate with binary search on a list.
9. **Graph DFS with large input:** Use iterative DFS with explicit stack to avoid recursion limit hits.
10. **`is` vs `==`:** `is` checks identity, `==` checks value. For node comparisons use `is`; for values use `==`.

Python Speed-Up Tips for Competitive Programming

1. Use `sys.stdin.readline` instead of `input()` for fast I/O.
2. Prefer local variables over global in tight loops (Python looks up locals faster).
3. Use `@cache` or `@lru_cache` for memoisation instead of manual dict.
4. `heapq.heapify` is $O(n)$; don't push one-by-one when building from scratch.
5. `bisect.bisect_left/right` for $O(\log n)$ search on sorted lists.
6. Use `collections.Counter` for frequency maps — faster than manual dicts.
7. Avoid `list.sort()` on already-nearly-sorted data with large k -inversions.
8. Use `frozenset` as dict key when you need a set as a hash key.
9. For set-like ordered operations, use `sortedcontainers.SortedList`.
10. Use `math.comb(n, r)` for exact nCr (no float rounding).